Universidad Técnica Nacional
Sede San Carlos
ISW-622 Pruebas de Software · Grupo 02

Investigación:

Casos de prueba, pruebas unitarias y TDD

¿Conviene aplicar TDD en todos los proyectos de software?

Integrantes
Valery Salas Vargas
Maykel Meoño González

Facilitador
Andrés Joseph Jiménez Leandro

sábado, 14 de noviembre del 2026

Índice

Resumen ejecutivo.....4
Abstract.....5
Introducción.....5
Contexto.....5
Pregunta central.....6
Objetivos.....6
Alcance.....6
Organización del informe.....7
Proceso de planificación.....7
División del trabajo.....7
Forma de trabajo.....7
Cronograma.....8
Registro de responsabilidades.....8
Riesgos identificados.....10
Metodología.....11
Enfoque.....11
Pasos.....11
Resumen.....14
Desarrollo.....14
Desarrollo A · Del requisito al caso de prueba.....14
Desarrollo B · Pruebas unitarias y TDD.....20
Postura condicionada sobre TDD.....26
Conclusiones.....29
Qué logró la investigación.....29
Respuesta a la pregunta central.....29

Qué cambia en el razonamiento del equipo....30
Por qué más pruebas no equivale automáticamente a mejor calidad....30
Límites de la evidencia....30
Pendientes y qué podría mejorarse....31
Lecciones aprendidas....31
Lecciones de Valery Salas Vargas....31
Lecciones de Maykel Meoño González....32
Referencias....33
Declaración de uso de inteligencia artificial....34
Anexos....35
Anexo A. Matriz de lectura crítica....35
Anexo B. Registro de responsabilidades....42
Anexo C. Bitácora de uso de IA....43
Anexo D. Figuras y su fuente....45
Anexo E. Código y bitácora de la demostración....46

Resumen ejecutivo

Este informe analiza los casos de prueba, las pruebas unitarias y el desarrollo guiado por pruebas (TDD) para responder la pregunta central del Tema 2: **¿es razonable defender TDD en todos los proyectos o su utilidad depende del problema, el equipo y los requisitos?** Se pretende entender cómo un requisito se transforma en una comprobación útil y evaluar críticamente el valor y los límites de las pruebas unitarias y de TDD.

La investigación es documental y cualitativa. Se basa en ocho fuentes:

- normas y documentos de organismos internacionales: el programa ISTQB de nivel básico y la norma ISO/IEC/IEEE 29119-1;
- un libro de texto de ingeniería del software: Pressman;
- documentación técnica y artículos de referencia profesional: JUnit 5, Agile Alliance, Vocke y Fowler;
- un estudio empírico sobre el proceso de TDD: Fucci et al., 2017.

Los conceptos se aplicaron al caso de la guía: un equipo recibe el requisito «el sistema aplicará descuentos correctos a los clientes frecuentes» y un desarrollador lo da por terminado con una sola prueba que confirma un 10 %. El análisis tiene dos partes:

- **Casos de prueba.** Se mostró que la cadena requisito → criterio de aceptación → caso de prueba → resultado esperado se rompe porque falta el criterio. Se identificaron diez ambigüedades, se separó la información que puede deducirse de la que debe preguntarse y, con reglas supuestas, se diseñaron doce casos normales, de límite y de error. La prueba original equivale a uno de ellos.
- **Pruebas unitarias y TDD.** Se describieron la prueba unitaria, el aislamiento, las aserciones, la estructura Arrange–Act–Assert, el ciclo rojo–verde–refactor y los riesgos de las pruebas frágiles y de la falsa sensación de seguridad. El ciclo se recorrió con un ejemplo pequeño en Java. Al introducir a propósito el error de usar **>** en lugar de **>=**, solo la prueba del valor exacto de la frontera lo detectó.

La conclusión es condicionada: TDD no puede defenderse en todos los proyectos. Conviene cuando hay reglas de negocio claras, cuando el código se va a mantener y refactorizar y cuando el equipo puede trabajar en pasos pequeños. Aporta poco en prototipos desechables, cuando el requisito todavía se está explorando o cuando hay que validar la integración, la interfaz o el rendimiento. Lo que sí aplica siempre es aclarar el requisito antes de probar: más pruebas sobre un requisito ambiguo no equivalen a más calidad. El estudio empírico consultado no encontró un efecto importante del orden en que se escriben las pruebas. Además, la mayor parte de la evidencia es normativa o de experiencia profesional, por lo que la conclusión no se generaliza.

Los resultados sirven a equipos de desarrollo y de pruebas para decidir cuándo adoptar TDD y para evaluar si una prueba realmente demuestra que un requisito se cumplió.

Abstract

This report examines test cases, unit tests, and test-driven development (TDD) in order to answer one question: should TDD be applied in every software project? The study is documentary and qualitative, and it draws on eight sources, including the ISTQB Foundation Level syllabus, the JUnit 5 user guide, and an empirical study of the TDD process. The concepts are applied to a case from the course guide, a requirement that grants discounts to frequent customers, which is analysed in two parts. The first part traces the path from requirement to acceptance criterion, test case, and expected result. It identifies ten ambiguities in the requirement and, under assumptions declared by the team, designs twelve test cases, eleven at unit level and one at interface level, using equivalence partitioning, boundary value analysis, and error guessing. The second part describes unit tests, isolation, assertions, and the Arrange–Act–Assert structure. It walks through a red–green–refactor cycle on a small Java example, whose red phases were observed with a minimal harness and whose final nine-test suite passes under JUnit 5, and discusses brittle tests and the false sense of security that a passing test suite can create. The conclusion is conditional. TDD is worthwhile when business rules are clear, the code will be maintained and refactored, and the team can work in small, uniform steps. It is less suitable for disposable prototypes, for requirements that are still being explored, or when integration, interface, or performance must be validated. The empirical study suggests that the benefit may come from working in small, uniform steps rather than from writing the test first. However, most of the evidence is normative or based on professional experience, so the conclusion cannot be generalised.

Keywords: test cases, unit testing, test-driven development, JUnit 5, boundary value analysis

Introducción

Contexto

Un defecto de software se vuelve más caro de corregir cuanto más tarde se detecta. Por eso la ingeniería del software advierte que las pruebas no deben verse como una «red de seguridad» al final del desarrollo: «la calidad se incorpora en el software a lo largo de todo el proceso de ingeniería del software» (Pressman, 2010, p. 385). Hay un estándar internacional, la serie ISO/IEC/IEEE 29119, cuya primera parte «specifies general concepts in software testing» (International Organization for Standardization [ISO] et al., 2022). El ISTQB, por su parte, define el vocabulario y las técnicas que usa la industria para diseñar pruebas (ISTQB, 2024).

Este informe trabaja con tres conceptos. El primero es el **caso de prueba**, que traduce un requisito y su criterio de aceptación en entradas concretas con un resultado esperado verificable. El segundo es la **prueba unitaria**, que el ISTQB define como la prueba que «focuses on testing components in isolation» (2024, p. 29). El tercero es el **desarrollo guiado por pruebas (TDD)**, que lleva la idea más lejos: la prueba se escribe antes que el código y «directs the coding through

test cases» (ISTQB, 2024, p. 26). TDD aplica el principio de probar temprano, o *shift left* (ISTQB, 2024, p. 26).

En la práctica, adoptar TDD tiene costos. Hay que escribir y mantener las pruebas, el equipo necesita disciplina y, si las pruebas están mal diseñadas, pueden romperse con cada cambio o dar una falsa sensación de seguridad. Por eso, en un equipo de desarrollo no se discute si probar o no, sino **cuánto y cuándo** conviene probar primero.

El tema es relevante para la calidad porque una prueba solo aporta evidencia si se deriva de un requisito claro. El caso que plantea la guía lo muestra: un equipo recibe el requisito «el sistema aplicará descuentos correctos a los clientes frecuentes», un desarrollador escribe una sola prueba que confirma un 10 % y da la funcionalidad por terminada, y otra persona propone aclarar las reglas antes de escribir más pruebas. La investigación usa ese caso para analizar qué hace útil a una prueba y cuándo conviene escribirla primero.

Pregunta central

> **¿Es razonable defender TDD en todos los proyectos o su utilidad depende del problema, el equipo y los requisitos?**

Objetivos

Objetivo general: determinar si TDD puede defenderse en todos los proyectos o en qué condiciones de problema, equipo y requisitos resulta útil. Para eso se analiza cómo un requisito se transforma en una comprobación útil, con base en fuentes normativas, técnicas y académicas.

Objetivos específicos:

1. Explicar la cadena requisito → criterio de aceptación → caso de prueba → resultado esperado y la trazabilidad entre sus elementos. *(Desarrollo A)*
2. Identificar las ambigüedades del requisito de descuentos a clientes frecuentes, la información que debe deducirse o preguntarse y los casos normales, límite y de error necesarios para juzgar si se cumplió. *(Desarrollo A)*
3. Describir la prueba unitaria (aislamiento, aserciones y la estructura Arrange–Act–Assert), el ciclo rojo–verde–refactor, qué pretende prevenir y qué no garantiza, y los riesgos de las pruebas frágiles. *(Desarrollo B)*
4. Formular una postura condicionada sobre TDD que diga cuándo sí y cuándo no conviene, qué cambia en el razonamiento del equipo y por qué más pruebas no equivale automáticamente a mejor calidad. *(Conclusión)*

Alcance

Incluye:

- casos de prueba y técnicas de caja negra para diseñarlos (partición de equivalencia, valores límite y conjetura de errores);

- pruebas unitarias, ilustradas con JUnit 5;
- TDD y su relación con ATDD.

Se toma como caso de estudio la situación de la ficha del Tema 2: el requisito de descuentos a clientes frecuentes.

No incluye:

- pruebas de integración, de sistema, de rendimiento y de seguridad, que solo se mencionan para ubicar las pruebas unitarias dentro de los niveles de prueba;
- la medición de la cobertura de código con herramientas;
- la comparación de *frameworks* de pruebas.

Límites de la evidencia: la investigación es documental. Las fuentes principales son normativas y de experiencia profesional; solo una aporta evidencia empírica (Fucci et al., 2017), y la demostración práctica es pequeña e ilustrativa. Por eso las conclusiones sobre TDD se presentan como condicionadas, no como generalizables a todos los proyectos.

Organización del informe

1. La **metodología** describe los siete pasos de la investigación.
2. El **Desarrollo A** trata los casos de prueba y su trazabilidad, y responde las preguntas de debate 1 y 2 sobre el caso de los clientes frecuentes.
3. El **Desarrollo B** trata las pruebas unitarias y el ciclo de TDD, y responde la pregunta de debate 3.
4. Al final están la postura sobre TDD, las conclusiones y las lecciones aprendidas.

Proceso de planificación

División del trabajo

El informe y la exposición del Tema 2, «Casos de prueba, pruebas unitarias y TDD», se dividieron por mitades entre Valery Salas Vargas (A) y Maykel Meoño González (B). Valery llevó las secciones que plantean el problema: resumen, introducción, metodología, requisitos y casos de prueba. Maykel llevó las que lo resuelven técnicamente (pruebas unitarias y TDD) y el formato Word. La postura final sobre TDD y las lecciones aprendidas se escribieron entre los dos. Cada sección tiene un responsable y la otra persona la revisa.

El reparto se acordó el 1 de noviembre de 2026 y la entrega es el 14 de noviembre de 2026.

Forma de trabajo

- Cada tarea se hace en una rama propia del repositorio compartido (por ejemplo `valery/tarea-3-metodologia`).
- Se abre un *pull request* hacia `main` con la otra persona como revisora. Los comentarios del PR son la constancia de la revisión cruzada. En los primeros PR hubo fusiones antes de la aprobación

de la otra persona; una ronda de correcciones (PR #3) arregló los errores que la revisión encontró después.

- Cada persona verifica las fuentes que cita y declara su uso de IA en la bitácora de anexos (Anexo C).
- Antes de la entrega, cada integrante practicará explicar el bloque del otro (ensayo del viernes 13 de noviembre), porque el profesor puede preguntar a cualquiera.

Cronograma

Fecha	Hito
Mar 3 nov	Fuentes verificadas (tareas 1 y 2)
Mié 4 nov	Metodología y proceso de planificación (tareas 3 y 4)
Sáb 7 nov	Matriz de lectura crítica e introducción (tareas 5 y 6)
Lun 9 nov	Desarrollo A y Desarrollo B (tareas 7 y 8)
Mar 10 nov	Postura sobre TDD, resumen, abstract y conclusiones (tareas 9 a 12)
Mié 11 nov	Lecciones aprendidas, referencias, anexos y diapositivas (tareas 13 a 15)
Jue 12 nov	Formato Word, tabla de correspondencia y PDF (tareas 16 y 17)
Vie 13 nov	Ensayo cronometrado de 15 + 10 minutos
Sáb 14 nov	Entrega y exposición (grupo 02, segundo en exponer)

Registro de responsabilidades

Los estados corresponden al **13 de noviembre de 2026**, según lo que consta en el repositorio.

#	Tarea	Responsable	Revisa	Fecha límite	Estado
1	Buscar y verificar 3 fuentes: ISTQB CTFL v4.0, ISO/IEC/IEEE 29119, Pressman	Valery	Maykel	mar 3 nov	Hecha
2	Buscar y verificar 3 fuentes: JUnit 5 User Guide, TDD (Agile Alliance), Vocke *Practical Test Pyramid*	Maykel	Valery	mar 3 nov	Hecha. Citas contrastadas con los originales en línea el 11-nov. Fucci et al. se contrastó con el preprint de arXiv, no con IEEE Xplore
3	Metodología (7 pasos, con instrumentos y criterios)	Valery	Maykel	mié 4 nov	Hecha

#	Tarea	Responsable	Revisa	Fecha límite	Estado
4	Proceso de planificación	Maykel	Valery	mié 4 nov	Hecha (estados actualizados el 13-nov)
5	Matriz de lectura crítica	Ambos	Ambos	sáb 7 nov	Hecha. Incluye la tabla en el formato de la guía
6	Introducción	Valery	Maykel	sáb 7 nov	Hecha
7	Desarrollo A	Valery	Maykel	lun 9 nov	Hecha
8	Desarrollo B	Maykel	Valery	lun 9 nov	Hecha (ajustada al caso de clientes frecuentes el 13-nov). El ciclo se observó con un arnés mínimo en Java; el 13-nov `mvn test` pasó las 9 pruebas con JUnit 5.9.2
9	Postura condicionada sobre TDD	Ambos	Ambos	mar 10 nov	Hecha. Está en el informe con los argumentos de ambos
10	Resumen ejecutivo	Valery	Maykel	mar 10 nov	Hecha
11	Abstract	Maykel	Valery	mar 10 nov	Hecha (cifras alineadas con el resumen el 13-nov)
12	Conclusiones	Valery	Maykel	mar 10 nov	Hecha
13	Lecciones aprendidas	Ambos	Ambos	mié 11 nov	Hecha. Cada integrante escribió las de los pasos que ejecutó
14	Referencias APA y declaración de uso de IA	Valery	Maykel	mié 11 nov	Hecha
15	Anexos	Maykel	Valery	mié 11 nov	Hecha. Anexos A a E
16	Formato Word	Maykel	Valery	jue 12 nov	Hecha el 13-nov: portada, índice automático, encabezado y número de página sin numerar la portada

#	Tarea	Responsable	Revisa	Fecha límite	Estado
17	Tabla de correspondencia y PDF	Ambos	Ambos	jue 12 nov	En curso: falta la revisión conjunta del PDF final

Presentación

Diapositivas	Contenido	Expone	Arma	Minutos
1–3	Portada, problema y pregunta central	Valery	Valery	2
4–7	Requisito, criterio, caso de prueba y trazabilidad; cobertura	Valery	Valery	4
8–11	Unitarias, AAA, rojo–verde–refactor, pruebas frágiles	Maykel	Maykel	4
12–13	Caso del descuento del 10 %: ambigüedades y preguntas	Valery	Valery	2
14–15	Postura condicionada sobre TDD, límites y contraargumento	Maykel	Maykel	2
16	Conclusión y pregunta final	Ambos	Ambos	1

La actividad práctica de 10 minutos la diseña Maykel y la conducen ambos. La lista de preguntas posibles del profesor la preparan ambos, la mitad cada uno.

Riesgos identificados

Riesgo	Medida
Tareas vencidas antes de la revisión cruzada (fuentes y planificación)	Hacer las revisiones en el mismo día en que se abre el PR
Fuente citada con autor incorrecto (*Practical Test Pyramid* es de Vocke, no de Fowler)	Verificar autor, fecha y versión contra el original antes de citar
El ciclo de la demostración no se observó con JUnit	Las fases en rojo se observaron con un arnés mínimo y así se declara en el Desarrollo B. La suite final se ejecutó con `mvn test` el 13-nov y pasó las 9 pruebas
Citas verificadas solo en versiones resumidas de las páginas	Maykel abre cada original antes de pasar el texto al Word (regla 2)
Nombre del archivo final (`.docx` en el patrón, pero se pide PDF)	Confirmar con el profesor

Metodología

Enfoque

La investigación es documental, con enfoque cualitativo y alcance descriptivo-analítico. Parte de una revisión de fuentes normativas y académicas sobre pruebas de software. Lo que se obtiene de ellas se aplica a la situación de la ficha del Tema 2, el requisito «el sistema aplicará descuentos correctos a los clientes frecuentes», para responder la pregunta central: **¿es razonable defender TDD en todos los proyectos o su utilidad depende del problema, el equipo y los requisitos?**

El trabajo se hizo en siete pasos. Cada uno tiene un instrumento, un criterio que indica cuándo el paso está completo y una persona responsable. Así se puede rastrear qué hizo cada integrante, y las lecciones aprendidas se escriben por paso.

Pasos

Paso 1. Delimitación del problema y de la pregunta central

Se analizó la guía del Tema 2 para definir la pregunta central, los objetivos y el alcance del informe. El alcance quedó en tres temas: casos de prueba, pruebas unitarias y desarrollo guiado por pruebas (TDD). Quedan fuera las pruebas de integración, de sistema y de rendimiento, salvo para ubicar las unitarias dentro de los niveles de prueba.

- **Instrumento:** guía del Tema 2 y un mapa de conceptos (requisito, criterio de aceptación, caso de prueba, prueba unitaria, TDD).
- **Criterio de cumplimiento:** la pregunta se puede responder con evidencia de las fuentes y cada objetivo corresponde a una sección del informe.
- **Responsable:** Valery y Maykel.

Paso 2. Búsqueda y selección de fuentes

Se seleccionaron ocho fuentes: tres a cargo de cada integrante, una fuente empírica acordada entre ambos y una fuente de apoyo. Se dio prioridad a organismos de estandarización y a autores de referencia en la disciplina.

- **Instrumento:** ficha bibliográfica por fuente, con la referencia APA 7, la versión o edición, el acceso al original y las secciones que respaldan cada parte del informe (carpeta `fuentes/` del repositorio).
- **Criterios de inclusión:**
  - autoridad: organismo de estandarización, autor o comunidad reconocida;
  - pertinencia directa con los tres temas;
  - vigencia: se usa la versión más reciente disponible;
  - acceso al documento original.
- **Criterios de exclusión:** Wikipedia, blogs sin autoría identificable y fuentes secundarias que solo resumen a otras.

- **Fuentes seleccionadas:**
  - Valery: ISTQB (2024), ISO/IEC/IEEE 29119-1 (2022) y Pressman (2010).
  - Maykel: JUnit 5 User Guide (Bechtold et al., s. f.), Agile Alliance (s. f.) sobre TDD y Vocke (2018), *The practical test pyramid*.
  - Acordada entre ambos (fuente 3 de la matriz): Fucci et al. (2017), la única con evidencia empírica sobre TDD.
  - De apoyo: Fowler (2023), usada solo para describir el ciclo rojo–verde–refactor.
- **Responsable:** cada fuente la busca y la verifica quien la cita.

Paso 3. Verificación y lectura crítica

Cada fuente se leyó en su versión original. Para cada afirmación que se usa en el informe se comprobó que la fuente realmente la respalda y se anotó la cita y la página.

- **Instrumento:** matriz de lectura crítica, con estas columnas: fuente, tipo de evidencia (normativa, experiencia profesional o empírica), afirmación principal, cita y página, fortalezas, limitaciones o sesgos, y uso en el informe.
- **Criterio de cumplimiento:** ninguna afirmación del informe queda sin una fuente verificada, y se distingue lo que una fuente *establece* (norma) de lo que *recomienda* (opinión experta).
- **Responsable:** fuente 1, Valery; fuente 2, Maykel; fuente 3, ambos.

Paso 4. Análisis del caso de los clientes frecuentes

Los conceptos de las fuentes se aplicaron al caso de la guía: un requisito de descuentos a clientes frecuentes que un desarrollador da por cumplido con una sola prueba del 10 %. Primero se identificaron las ambigüedades que impiden juzgar esa prueba. Después se separó la información que puede deducirse de la que debe preguntarse. Por último, con reglas supuestas por el equipo y declaradas como tales, se reconstruyó la cadena requisito → criterio de aceptación → caso de prueba → resultado esperado.

Los casos se diseñaron con dos técnicas de caja negra descritas en el programa ISTQB (2024, secc. 4.2.1 y 4.2.2):

- **Partición de equivalencia:** una prueba por partición válida e inválida.
- **Análisis de valores límite:** se evalúan los valores en cada frontera de las particiones.
- **Instrumento:** matriz de trazabilidad (ID del requisito, criterio de aceptación, ID del caso, tipo de caso, datos de entrada, resultado esperado) una lista de ambigüedades y una tabla de información por deducir o preguntar, con la fuente o persona a quien consultar.
- **Criterio de cumplimiento:**
  - cada criterio de aceptación tiene al menos un caso;
  - cada caso se puede trazar a un criterio (ISTQB, 2024, secc. 1.4.4);
  - el conjunto cubre casos normales, límites y de error.

- **Responsable:** Valery.

Paso 5. Demostración del ciclo de TDD

Para observar el ciclo rojo–verde–refactor, los casos del paso 4 se automatizaron como pruebas unitarias con JUnit 5. No se pudo descargar JUnit en el entorno de preparación. Por eso cada fase se ejecutó con un arnés mínimo en Java, con las mismas entradas, valores esperados y tolerancia que las pruebas. Después, la suite final se ejecutó con `mvn test` (JUnit 5.9.2) el 13 de noviembre de 2026 y las 9 pruebas pasaron. Las pruebas siguen la estructura Arrange–Act–Assert. Se registró qué pasó en cada fase: la prueba falla antes de existir el código, pasa con la implementación mínima y se mantiene en verde al refactorizar. También se anotaron las pruebas que se rompieron por cambios internos que no alteraban el comportamiento, es decir, las pruebas frágiles.

- **Instrumento:** proyecto de ejemplo con JUnit 5 (suite final ejecutada con `mvn test`), un arnés mínimo en Java y una bitácora del ciclo (prueba, fase, resultado, observación) en el Anexo E.
- **Criterio de cumplimiento:** se ve al menos un ciclo completo rojo–verde–refactor y cada prueba se puede relacionar con un caso de la matriz de trazabilidad.
- **Responsable:** Maykel.

Paso 6. Síntesis y postura condicionada sobre TDD

Se compararon los argumentos de las fuentes y lo observado en los pasos 4 y 5. Con eso se formuló una postura condicionada: en qué contextos TDD conviene y en cuáles no. La postura incluye su contraargumento principal y los límites de la evidencia.

- **Instrumento:** tabla de argumentos, con las columnas condición del proyecto, argumento a favor, argumento en contra, fuente o evidencia y peso.
- **Criterio de cumplimiento:**
  - la postura se apoya en al menos dos fuentes independientes;
  - presenta y responde un contraargumento;
  - dice qué evidencia nueva haría cambiar la conclusión, que es la pregunta final de la exposición.
- **Responsable:** Valery y Maykel.

Paso 7. Revisión cruzada y control de calidad

Cada sección la revisó el integrante que no la escribió. Las revisiones se hicieron por *pull request* en el repositorio compartido, y los comentarios quedaron registrados como constancia de que ambos conocen todo el informe. Antes de exportar el PDF se verificó el informe contra la tabla de control de correspondencia de la guía.

- **Instrumento:** *pull requests* en GitHub con revisor asignado, la tabla de control de correspondencia y la bitácora de uso de IA en los anexos.
- **Criterio de cumplimiento:**

  - toda sección tiene una revisión aprobada;
  - cada requisito de la guía aparece en el informe;
  - las referencias están en APA 7;
  - el uso de IA de cada integrante está declarado.
- **Responsable:** Valery y Maykel.

Resumen

Paso	Instrumento	Criterio de cumplimiento	Responsable
1. Delimitación	Guía y mapa de conceptos	Pregunta respondible con evidencia	Ambos
2. Búsqueda y selección	Ficha bibliográfica	Inclusión y exclusión aplicadas; 8 fuentes	Cada quien las suyas
3. Lectura crítica	Matriz de lectura crítica	Toda afirmación verificada con cita y página	V: fuente 1 · M: fuente 2 · ambos: fuente 3
4. Análisis del caso	Matriz de trazabilidad	Cobertura de casos normales, límites y errores	Valery
5. Demostración TDD	JUnit 5 y bitácora del ciclo	Ciclo rojo–verde–refactor completo	Maykel
6. Síntesis	Tabla de argumentos	Postura con 2 o más fuentes y contraargumento	Ambos
7. Revisión y control	PR, tabla de correspondencia y bitácora de IA	Todo revisado y correspondido	Ambos

Desarrollo

Desarrollo A · Del requisito al caso de prueba

Esta sección responde los dos primeros aspectos de la ficha del Tema 2 y las preguntas de debate 1 y 2. Explica cómo un requisito se transforma en una comprobación útil y lo aplica al caso de los descuentos a clientes frecuentes. Los casos diseñados aquí (CP-01 a CP-12) se retoman en el Desarrollo B. Allí, los casos CP-01 a CP-09 se automatizan como pruebas unitarias.

1. Requisito, criterio de aceptación, caso de prueba y resultado esperado: relaciones y trazabilidad

Elemento	Qué es	Pregunta que responde
Requisito	Lo que el sistema debe hacer, expresado por el cliente o el analista	¿Qué se necesita?
Criterio de aceptación	Condición concreta y verificable que el requisito debe cumplir para darse por aceptado	¿Cómo sabremos que está bien hecho?
Caso de prueba	Conjunto de precondiciones, datos de entrada y pasos que ejercitan un criterio	¿Con qué datos lo comprobamos?

Elemento	Qué es	Pregunta que responde
Resultado esperado	Salida o comportamiento que debe producirse con esos datos, definido *antes* de ejecutar	¿Qué debería pasar?

Los cuatro elementos forman una cadena: cada uno se deriva del anterior. El criterio de aceptación es el eslabón que vuelve verificable al requisito. Según el ISTQB, sirve para «Define the scope of the user story» y para «Serve as a basis for the user story acceptance testing» (2024, p. 45–46). Un caso de prueba aparece cuando una condición de prueba se concreta con datos, actividad que el ISTQB describe como «elaborating the test conditions into test cases» (2024, p. 19). El resultado esperado tiene que salir del criterio y fijarse antes de ejecutar. Si no, cualquier salida parece correcta.

Trazabilidad. Es la relación documentada entre los eslabones de la cadena. Según el ISTQB (2024, p. 20), «it is important to establish and maintain traceability throughout the test process between the test basis elements» y los productos de la prueba. En la práctica permite responder tres preguntas:

1. **Cobertura:** ¿todo criterio de aceptación tiene al menos un caso?
2. **Impacto:** si el requisito cambia, ¿qué casos hay que actualizar?
3. **Justificación:** ¿cada caso existe por una razón, o hay casos que no verifican nada pedido?

Aplicado al caso. Al requisito «el sistema aplicará descuentos correctos a los clientes frecuentes» le falta el eslabón del criterio de aceptación. Por eso la única prueba del desarrollador no se puede trazar a nada verificable: confirma un 10 % que nadie definió como el resultado esperado. La cadena se rompe en el segundo eslabón, y sin criterio no hay forma de decidir si una prueba, o cien, son suficientes. La sección 3 reconstruye la cadena completa.

2. Cobertura de casos normales, límites y errores; razones para evitar pruebas que solo confirman el caso feliz

2.1 Por qué una prueba del caso feliz no basta

La prueba del desarrollador comprueba un solo valor de una sola situación: un cliente frecuente que recibe el 10 %. Esa prueba seguiría pasando con implementaciones claramente incorrectas. Algunos ejemplos:

- el sistema da el 10 % a **todos** los clientes, frecuentes o no;
- el sistema nunca deja de considerar frecuente a un cliente;
- el sistema calcula mal el descuento en la frontera de la regla, por ejemplo con `>` en lugar de `>=`;
- el sistema acepta montos negativos.

Una prueba que no puede fallar ante esos errores no aporta evidencia de que el requisito se cumpla. Solo confirma lo que el desarrollador ya creía. Pressman describe ese sesgo: quien construye el software tiende a diseñar «pruebas que demostrarán que el programa funciona, en

lugar de descubrir errores» (2010, p. 385). La demostración del Desarrollo B lo muestra: al cambiar a propósito `>=` por `>`, la prueba de 99,99 siguió pasando y solo la del valor exacto de la frontera detectó el error.

Para evitarlo, el ISTQB propone técnicas que derivan los casos del requisito y no de la intuición:

- **Partición de equivalencia:** divide las entradas en grupos que el sistema debe tratar igual; con un caso por grupo basta, porque «one test for each partition is sufficient» (2024, p. 39).
- **Análisis de valores límite:** prueba cada frontera entre grupos y sus vecinos, ya que en las fronteras «developers are more likely to make errors» (ISTQB, 2024, p. 40–41). Pressman coincide: «un mayor número de errores ocurre en las fronteras del dominio de entrada y no en el "centro"» (2010, p. 425).
- **Conjetura de errores:** anticipa fallos típicos a partir de la experiencia (ISTQB, 2024, p. 43).

Con estas técnicas, la cobertura se mide contra los criterios, las particiones y las fronteras, no por la cantidad de pruebas.

2.2 Casos diseñados para la regla aclarada

Los casos solo se pueden diseñar después de responder las preguntas de la sección 3. Para continuar el análisis, el equipo **supone** las respuestas que aparecen en la sección 3.3. Son supuestos propios, no datos de la guía, y su único fin es mostrar cómo se completa la cadena.

Particiones y fronteras

Variable	Particiones	Fronteras probadas
Compras del cliente en los últimos 6 meses	< 5: no frecuente · ≥ 5: frecuente	4 y 5
Monto de la compra (cliente frecuente)	< 0: inválido · 0 a 99,99: sin descuento · ≥ 100: 10 %	−0,01 · 0 · 0,01 · 99,99 · 100 · 100,01

En la frontera de 100 se usa el análisis de 3 valores (100 y sus dos vecinos), porque ahí está el error más probable. En las demás se usan 2 valores.

Matriz de trazabilidad

Criterio	Caso	Tipo	Técnica	Precondición	Entrada	Resultado esperado
CA-01	**CP-01**	Normal	Partición	Cliente frecuente	50,00	50,00
CA-02	**CP-02**	Límite	Valor límite (100)	Cliente frecuente	100,00	90,00
CA-01	**CP-03**	Límite	Valor límite (100)	Cliente frecuente	99,99	99,99

Criterio	Caso	Tipo	Técnica	Precondición	Entrada	Resultado esperado
CA-03	**CP-04**	Error	Partición	Cliente frecuente	−1,00	Error: «El monto no puede ser negativo»
CA-02	**CP-05**	Normal	Partición	Cliente frecuente	200,00	180,00
CA-01	**CP-06**	Límite	Valor límite (0)	Cliente frecuente	0,00	0,00
CA-02, CA-04	**CP-07**	Límite	Valor límite (100, 3 valores)	Cliente frecuente	100,01	90,01
CA-03	**CP-08**	Límite y error	Valor límite (0)	Cliente frecuente	−0,01	Error
CA-01	**CP-09**	Límite	Valor límite (0)	Cliente frecuente	0,01	0,01
—	CP-10	Error	Conjetura de errores	—	vacío / «abc»	Mensaje de validación (nivel de interfaz)
CA-05	**CP-11**	Límite	Valor límite (frecuencia)	4 compras en 6 meses	200,00	200,00 (sin descuento)
CA-06	**CP-12**	Límite	Valor límite (frecuencia)	5 compras en 6 meses	200,00	180,00

Comprobación de cobertura

Criterio	Casos	¿Cubierto?
CA-01	CP-01, CP-03, CP-06, CP-09	Sí
CA-02	CP-02, CP-05, CP-07	Sí
CA-03	CP-04, CP-08	Sí
CA-04	CP-07	Sí, con el supuesto de redondeo
CA-05	CP-11	Sí
CA-06	CP-12	Sí

Tipo	Cantidad	Casos
Normales	2	CP-01, CP-05
Límites	7	CP-02, CP-03, CP-06, CP-07, CP-09, CP-11, CP-12
Límite y error a la vez	1	CP-08
Errores	2	CP-04 · CP-10 (otro nivel)
Total	**12**	11 unitarios + 1 de interfaz

La prueba original del desarrollador equivale a CP-05: es uno de doce casos. Los casos de límite y de error son la mayoría, y son los que detectan los errores que el caso feliz deja pasar.

> **Relación con el Desarrollo B.** CP-01 a CP-09 están automatizados en `demo-tdd/` para un cliente ya clasificado como frecuente, cada prueba con el ID de su caso. CP-11 y CP-12, la clasificación por frecuencia, están diseñados pero no automatizados: necesitarían una segunda unidad que cuente las compras del cliente. CP-10 corresponde al nivel de interfaz.

3. Situación para el análisis crítico

> *Un equipo recibe el requisito «el sistema aplicará descuentos correctos a los clientes frecuentes». Un desarrollador crea una sola prueba que confirma un descuento del 10 % y declara finalizada la funcionalidad. Otra persona propone aclarar las reglas antes de escribir más pruebas.*

3.1 Pregunta de debate 1: ¿Qué ambigüedades impiden juzgar si la prueba es suficiente?

Para juzgar si una prueba es suficiente hace falta saber contra qué se compara. El requisito no lo dice: tiene dos términos sin definir, «clientes frecuentes» y «descuentos correctos», y varias condiciones implícitas.

#	Ambigüedad	Por qué impide juzgar la prueba
A1	**¿Quién es «cliente frecuente»?** ¿Cantidad de compras, monto acumulado, antigüedad o membresía? ¿En qué período?	Sin esta definición no se sabe a quién debe aplicarse el descuento, ni a quién no. La prueba no verifica a ningún cliente no frecuente
A2	**¿Dónde está la frontera de la frecuencia?** Si son 5 compras, ¿5 exactas cuentan?	Es el error de frontera más probable y la prueba no lo toca
A3	**¿Qué es un descuento «correcto»?** ¿10 % fijo, escalonado por nivel de cliente o dependiente del monto?	El 10 % que confirma la prueba es una suposición del desarrollador, no un criterio del cliente
A4	**¿Hay un monto mínimo de compra?**	Si existe, el descuento no aplica siempre y faltan casos por debajo del mínimo
A5	**¿Sobre qué monto se calcula?** ¿Subtotal, con impuestos o con envío?	Cambia el resultado esperado de todos los casos
A6	**¿Cómo se redondea?**	100,01 × 0,9 = 90,009. Puede cobrarse 90,01 o 90,00
A7	**¿Se combina con otras promociones? ¿Hay un tope?**	Un cliente frecuente con otra promoción puede recibir un descuento doble o uno incorrecto
A8	**¿Hay productos o categorías excluidos?**	La prueba usa un solo producto o monto
A9	**¿Qué pasa con entradas inválidas o devoluciones?**	El requisito solo describe el caso feliz

#	Ambigüedad	Por qué impide juzgar la prueba
A10	**¿Cuándo se evalúa si el cliente es frecuente?** ¿Al momento de la compra? ¿Qué pasa si pierde la condición?	Afecta el resultado de compras hechas cerca del cambio de condición

Respuesta. Con esas ambigüedades, la prueba no se puede juzgar como suficiente ni como insuficiente, porque no existe un criterio contra el cual medirla. Lo único que se puede afirmar es que comprueba **un** comportamiento supuesto, un 10 % para un cliente que el desarrollador consideró frecuente. No cubre a ningún cliente no frecuente, ninguna frontera y ningún error. Declarar terminada la funcionalidad es una conclusión que la evidencia no sostiene. La propuesta de aclarar las reglas antes de escribir más pruebas es la correcta: escribir más pruebas sobre un requisito ambiguo solo multiplica la suposición inicial.

3.2 Pregunta de debate 2: ¿Qué información tendría que deducirse o preguntarse antes de decidir que el requisito se cumplió?

Parte de la información se puede **deducir** del contexto del sistema y otra parte solo la puede dar quien pidió el requisito. Lo importante es no confundir las dos: lo que se deduce también debe confirmarse antes de darlo por cierto.

Información	¿Deducir o preguntar?	Dónde buscarla o a quién preguntar	Qué decide
Definición de cliente frecuente (A1) y su frontera (A2)	**Preguntar**	Dueño del producto o área comercial	Particiones y fronteras de frecuencia (CA-05, CA-06)
Porcentaje o escala de descuento (A3)	**Preguntar**	Área comercial	Resultado esperado de todos los casos con descuento
Monto mínimo (A4) y base de cálculo (A5)	Deducir de las políticas de precios existentes, **y confirmar**	Documentación del negocio y dueño del producto	Fronteras de monto (CA-01, CA-02)
Redondeo y moneda (A6)	Deducir de cómo el sistema ya maneja montos y de la normativa de facturación, **y confirmar**	Código y facturas existentes; contabilidad	CA-04
Combinación con promociones y topes (A7)	**Preguntar**	Área comercial	Casos de interacción entre descuentos
Exclusiones (A8)	Deducir del catálogo, **y confirmar**	Catálogo de productos	Casos por categoría
Entradas inválidas y devoluciones (A9)	Deducir de las reglas de validación del sistema	Código y requisitos de validación	CA-03 y casos de error
Momento de evaluación de la frecuencia (A10)	**Preguntar**	Dueño del producto	Casos de cambio de condición

Respuesta. Antes de decidir que el requisito se cumplió hace falta, como mínimo:

- la definición verificable de «cliente frecuente» y de «descuento correcto», que solo el cliente puede dar;
- los criterios de aceptación escritos a partir de esas definiciones y aceptados por el cliente;
- un conjunto de casos trazado a esos criterios que cubra las particiones, las fronteras y los errores.

El requisito se considera cumplido cuando todos los casos trazados pasan, no cuando pasa una prueba. Si una respuesta del cliente cambia, la trazabilidad indica qué casos hay que rehacer.

3.3 Respuestas supuestas para continuar el análisis

Como no hay un cliente real a quien preguntar, el equipo supone estas respuestas para completar la cadena. Con cualquier otra respuesta cambian los criterios y los casos, pero el método es el mismo.

Ambigüedad	Supuesto del equipo
A1, A2	Cliente frecuente: 5 o más compras en los últimos 6 meses (5 exactas cuentan)
A3, A4	Un cliente frecuente recibe 10 % en compras de 100 o más. Debajo de 100, sin descuento
A5	Se calcula sobre el subtotal
A6	El total se redondea a dos decimales
A9	Los montos negativos son inválidos y se rechazan
A7, A8, A10	Fuera del alcance de este análisis: quedan como preguntas abiertas

Criterios de aceptación resultantes

ID	Criterio (*Dado / Cuando / Entonces*)
CA-01	**Dado** un cliente frecuente y un monto entre 0 y 99,99, **cuando** se calcula el total, **entonces** no se aplica descuento.
CA-02	**Dado** un cliente frecuente y un monto de 100 o más, **cuando** se calcula el total, **entonces** se descuenta el 10 %.
CA-03	**Dado** un monto negativo, **cuando** se calcula el total, **entonces** el sistema rechaza la operación.
CA-04	**Dado** cualquier total calculado, **entonces** se presenta redondeado a dos decimales.
CA-05	**Dado** un cliente con menos de 5 compras en los últimos 6 meses, **cuando** compra 100 o más, **entonces** no recibe descuento.
CA-06	**Dado** un cliente con 5 o más compras en los últimos 6 meses, **cuando** compra 100 o más, **entonces** recibe el 10 %.

Desarrollo B · Pruebas unitarias y TDD

Esta sección explica qué es una prueba unitaria, cómo se escribe y cómo se usa en el ciclo de TDD. Los ejemplos usan el caso de la guía, el requisito «el sistema aplicará descuentos correctos

a los clientes frecuentes», con las reglas que el equipo supuso en el Desarrollo A (10 % desde 100, redondeo a dos decimales, negativos inválidos). Se automatizan los casos CP-01 a CP-09 de su matriz de trazabilidad, que prueban el cálculo del total para un cliente **ya clasificado como frecuente**. La clasificación por frecuencia (CP-11 y CP-12) está diseñada, pero no automatizada. Las pruebas automatizadas están en el proyecto `demo-tdd/` (JUnit 5.9.2) y la bitácora del ciclo, en el Anexo E.

1. La prueba unitaria

El ISTQB (2024, p. 29) define la prueba de componente, también llamada prueba unitaria, como la que «focuses on testing components in isolation». Una prueba unitaria revisa una unidad pequeña de código (un método o una clase) sin depender del resto del sistema, y su resultado es automático: pasa o falla.

Vocke (2018) las ubica como la base de la estrategia de pruebas: «The foundation of your test suite will be made up of unit tests». Son rápidas, baratas de ejecutar y localizan el defecto con precisión. Por eso se ejecutan cientos de veces al día.

1.1 Aislamiento

Aislar significa que el resultado de la prueba depende solo de la unidad probada. Para lograrlo se reemplazan sus dependencias (base de datos, red, reloj, otras clases) por **dobles de prueba**. Vocke (2018) advierte que mucha gente usa *mock* y *stub* como sinónimos. Para este informe se usa esta distinción: un *stub* devuelve respuestas fijas y un *mock* además permite comprobar que se le hizo una llamada.

En el caso del descuento, la calculadora recibe el monto y no tiene dependencias externas, así que no necesita dobles. Eso la convierte en un buen primer ejemplo. El aislamiento real aparecería al probar la clasificación del cliente (CP-11 y CP-12): contar las compras de los últimos seis meses exige consultar un historial, y ahí se usaría un *stub* de ese repositorio que devuelva 4 o 5 compras fijas.

1.2 Aserciones

Una **aserción** compara el resultado real con el esperado. Si no coinciden, la prueba falla. JUnit 5 reúne las aserciones en la clase `Assertions` de Jupiter. Las dos que usa la demostración son `assertEquals` (valor esperado frente a valor real, con tolerancia para números decimales) y `assertThrows`, que verifica que un bloque de código lance una excepción concreta y sirve para los casos de error. La guía de JUnit muestra su uso en la sección de aserciones (Bechtold et al., s. f., secc. 2.5), con una excepción aritmética; la demostración lo aplica a `IllegalArgumentException` para CP-04 y CP-08.

JUnit 5 se compone de tres módulos: «JUnit Platform + JUnit Jupiter + JUnit Vintage» (Bechtold et al., s. f., secc. 1.1). Para escribir pruebas nuevas se usa Jupiter. Se marca un método como

prueba con `@Test` («Denotes that a method is a test method»), y la preparación común se pone en un método con `@BeforeEach`, que según la guía «should be executed before each @Test» (y antes de cada prueba repetida, parametrizada o de fábrica) de la clase (Bechtold et al., s. f., secc. 2.1). Así cada prueba empieza con un estado nuevo, y eso mantiene a las pruebas independientes entre sí.

1.3 Estructura Arrange–Act–Assert (AAA)

Vocke (2018) propone una estructura fija de tres pasos para toda prueba («Set up the test data», «Call your method under test» y «Assert that the expected results are returned») y la resume con el mnemónico «Arrange, Act, Assert»:

Fase	Qué hace	En el ejemplo
Arrange (preparar)	Crea los datos y el objeto bajo prueba	`double monto = 50.0;`
Act (ejecutar)	Llama una sola vez al método que se prueba	`calculadora.calcularTotal(monto)`
Assert (comprobar)	Compara el resultado con el esperado	`assertEquals(50.0, total, 0.001)`

Una prueba con una sola fase *Act* y pocas aserciones indica con claridad qué se rompió cuando falla. Cada caso del Desarrollo A se convierte en una prueba con este molde, y el nombre del método y `@DisplayName` llevan el ID del caso para conservar la trazabilidad:

```
@Test
@DisplayName("CP-02 límite: monto exacto en el umbral recibe 10 %")
void montoEnElUmbralRecibeDescuento() {
    double total = calculadora.calcularTotal(100.0);   // Act (Arrange es el
objeto del @BeforeEach)
    assertEquals(90.0, total, 0.001);                  // Assert
}
```

Para varios valores de una misma partición o frontera, `@ParameterizedTest` («Denotes that a method is a parameterized test») ejecuta el mismo método con distintos datos y evita copiar pruebas (Bechtold et al., s. f., secc. 2.1). La demostración la usa para CP-05, CP-06, CP-07 y CP-09, con el ID del caso en la primera columna de los datos.

2. El ciclo rojo–verde–refactor de TDD

El ISTQB (2024, p. 26) describe TDD como una práctica que «directs the coding through test cases» y que aplica el principio de probar temprano. Fowler (2023) resume el ciclo en tres pasos que se repiten: escribir una prueba para la siguiente funcionalidad, escribir el código que la hace pasar y refactorizar «both new and old code». Para la comunidad ágil, TDD es un estilo de programar en el que se entrelazan tres actividades: «coding, testing (in the form of writing unit tests), and design (in the form of refactoring)» (Agile Alliance, s. f.).

Fase	Qué se hace	Estado de las pruebas
🔴 **Rojo**	Se escribe una prueba para algo que aún no existe y se ejecuta. Debe fallar	Falla (si no falla, la prueba no aporta información)
🟢 **Verde**	Se escribe el código mínimo para que pase	Pasa
🔵 **Refactor**	Se mejora la estructura sin cambiar el comportamiento	Siguen pasando

Fowler (2023) señala que omitir el refactor es la forma más común de hacer mal TDD: sin él se acumula código que funciona pero está mal estructurado.

2.1 Un ciclo completo con el caso del descuento

La siguiente secuencia usa los casos de la matriz de trazabilidad del Desarrollo A. La columna «Resultado observado» sale de la ejecución del 11 de noviembre de 2026 (bitácora completa en el Anexo E y en `anexos/bitacora-ciclo-tdd.md`).

> **Cómo se ejecutó.** No se pudo descargar JUnit en el entorno de preparación, así que cada paso se corrió con un arnés mínimo en Java que usa las mismas entradas, los mismos valores esperados y la misma tolerancia que las pruebas. Eso comprueba la lógica del ciclo. Las fases en rojo se observaron con ese arnés. El 13 de noviembre de 2026 se ejecutó la suite final con `mvn test` (JUnit 5.9.2, Java 17.0.21): 9 pruebas ejecutadas, 0 fallas, 0 errores, 0 omitidas y BUILD SUCCESS. Esa ejecución confirma que las pruebas funcionan con JUnit, pero no repite las fases intermedias del ciclo.

Paso	Prueba	Fase	Código de producción	Resultado observado
1	CP-01 (50 → 50)	Rojo	La clase no existe	No compila: «cannot find symbol … class CalculadoraDescuento »
2	CP-01	Verde	`return monto;`	Pasa
3	CP-02 (100 → 90)	Rojo	Sigue `return monto;`	Falla: se obtiene 100,0 y se esperaba 90,0
4	CP-02	Verde	`return monto >= 100 ? monto * 0.9 : monto;`	CP-01 y CP-02 pasan
5	CP-03 (99,99 → 99,99)	Verde sin rojo	Sin cambios	**Pasa de inmediato.** No hubo fase roja
6	CP-04 (−1 → excepción)	Rojo	No valida negativos	Falla: no lanza excepción y devuelve −1,0
7	CP-04	Verde	Se agrega `if (monto < 0) throw …`	CP-01 a CP-04 pasan

Paso	Prueba	Fase	Código de producción	Resultado observado
8	CP-07 (100,01 → 90,01)	Rojo	Sin redondeo	Falla: se obtiene 90,009 y se esperaba 90,01
9	CP-07	Verde	`Math.round(total * 100.0) / 100.0`	CP-01 a CP-04 y CP-07 pasan
10	CP-01 a CP-09	Refactor	`100` y `0.10` pasan a constantes `UMBRAL` y `TASA_DESCUENTO`; el redondeo pasa a un método privado	Las 9 pruebas pasan

En el paso 10 se agregan CP-05, CP-06, CP-08 y CP-09. Pasan de inmediato, como CP-03: protegen el comportamiento, pero no se vieron fallar.

El paso 5 ilustra un punto importante: una prueba que nunca estuvo en rojo no demuestra que detecte el error que debería detectar. En TDD estricto, para comprobarlo se rompe el código a propósito. En la ejecución se cambió `>=` por `>` (mutante): CP-02 falló («obtuvo 100,0, esperaba 90,0») y CP-03 siguió pasando. Es decir, la frontera de 100 la protege CP-02 y no CP-03, que por sí sola no habría detectado ese error.

El paso 8 muestra además que una prueba puede forzar una decisión de requisito. Para escribir CP-07 hubo que decidir cómo se redondea 90,009, que es la ambigüedad A3 del Desarrollo A. La demostración supone dos decimales (criterio CA-04). Si el cliente responde otra cosa, cambian el código y la prueba.

2.2 Qué aporta el ciclo y qué no

Según la comunidad ágil, «many teams report significant reductions in defect rates» (Agile Alliance, s. f.), a cambio de un aumento moderado del esfuerzo inicial. Es un testimonio de equipos, y la entrada no cita estudios. Fucci et al. (2017) analizaron datos de 39 profesionales y encontraron que el orden de escritura (prueba antes o después del código) «had no important influence»; lo que se asociaba con mejores resultados eran los pasos pequeños y uniformes. Esto sugiere que el valor de TDD está más en el ritmo de trabajo que en el orden. Es una evidencia limitada (sin grupo de control, solo tres tareas, dos de ellas artificiales), por lo que se trata como una señal y no como una prueba. Se retoma en la postura final.

3. Pruebas frágiles

Una prueba es **frágil** cuando falla por cambios que no alteran el comportamiento observable. El síntoma es que al refactorizar se rompen pruebas aunque el sistema siga funcionando. Vocke (2018) da la regla para evitarlo: «Don’t reflect your internal code structure within your unit tests» y «Test for observable behaviour instead».

Ejemplo (explicado, no ejecutado). Suponga que el cálculo se separa en un método privado `aplicarTasa()`. Una prueba frágil verificaría con un *mock* que `aplicarTasa()` fue llamado una vez. Si en el refactor se elimina ese método y el cálculo se escribe en una sola línea, la prueba falla aunque `calcularTotal(100)` siga devolviendo 90. Una prueba robusta solo comprueba la entrada y la salida:

	Prueba frágil	**Prueba robusta**
Qué verifica	Que se llamó a `aplicarTasa()`	Que `calcularTotal(100.0)` devuelve 90
Se rompe al refactorizar	Sí	No
Se rompe cuando el cálculo es incorrecto	No necesariamente	Sí

Otras causas habituales son depender del orden de ejecución, de la hora del sistema o de datos compartidos entre pruebas. `@BeforeEach` ayuda a evitarlas porque cada prueba empieza con un objeto nuevo (Bechtold et al., s. f., secc. 2.1).

El costo es real: «Every single test in your test suite is additional baggage and doesn’t come for free» (Vocke, 2018). Las pruebas frágiles multiplican ese costo, porque se pasa más tiempo arreglándolas que protegiendo el código.

4. Falsa sensación de seguridad

Una suite en verde solo indica que **los casos escritos pasan**. No garantiza que el programa sea correcto, y puede dar confianza que no corresponde. Algunos ejemplos con el caso del descuento:

- **Casos que faltan.** Si no existe el caso de 100 exacto, el error `>` en lugar de `>=` pasa sin ser detectado (en la demostración, CP-03 pasa con ambos operadores). El ISTQB (2024, secc. 4.2.2) explica que en las fronteras es donde los desarrolladores tienen más probabilidad de equivocarse («developers are more likely to make errors with these boundary values»), y por eso hay que probarlas.
- **Aserciones débiles.** Una prueba que solo comprueba `assertNotNull(total)` pasa aunque el cálculo esté mal. Ejecutar el código sin verificar su resultado sube la cobertura, pero no aumenta la protección.
- **Prueba escrita después, a la medida del código.** Si se escribe mirando la implementación, tiende a repetir sus errores. En el ciclo rojo–verde la prueba se escribe antes, desde el requisito.
- **Requisito mal entendido.** Las pruebas pueden verificar con total precisión algo equivocado. Si «cliente frecuente» o «descuento correcto» significaban otra cosa, las pruebas en verde confirman un error (ver ambigüedades A1 a A3 del Desarrollo A).
- **Una parte del requisito sin pruebas.** CP-01 a CP-09 parten de un cliente ya clasificado como frecuente. Un error en la clasificación, por ejemplo contar con `> 5` en lugar de `>= 5` compras (ejemplo ilustrativo, no está en `demo-tdd/`), pasaría las nueve pruebas. Solo CP-12 (exactamente 5 compras) lo detectaría, y está diseñado pero no automatizado.

Para reducir este riesgo se combinan tres prácticas: derivar los casos del requisito con técnicas como partición y valores límite (ISTQB, 2024, secc. 4.2), verificar que cada prueba falle al menos una vez cuando el código está mal y mantener las pruebas pequeñas y legibles.

5. Pregunta de debate 3

¿Puede un equipo confiar en que su software está correcto si todas las pruebas unitarias pasan?

No del todo. Una suite en verde demuestra que los casos escritos pasan, no que no haya defectos. En la demostración, las nueve pruebas pasan y aun así la clasificación del cliente frecuente queda sin probar. Su valor depende de tres cosas: que los casos cubran normales, límites y errores; que las aserciones verifiquen resultados concretos y no solo que el código se ejecute; y que las pruebas comprueben comportamiento y no detalles internos, para que sigan siendo útiles tras refactorizar. Además, ninguna prueba unitaria detecta un requisito mal entendido, por lo que debe acompañarse de preguntas al cliente y de pruebas de otros niveles. El ISTQB (2024) pone las pruebas de componente en el primer nivel de una serie, no como sustituto de las demás.

Postura condicionada sobre TDD

Pregunta central

¿Es razonable defender TDD en todos los proyectos o su utilidad depende del problema, el equipo y los requisitos?

Respuesta corta

Su utilidad depende del problema, el equipo y los requisitos. TDD conviene cuando el comportamiento esperado es claro, el código va a cambiar y mantenerse durante tiempo, y el equipo puede sostener la disciplina de pruebas pequeñas. Conviene menos, o conviene otra práctica, cuando el requisito todavía se está explorando, cuando el código es descartable, o cuando lo que importa probar no es una unidad aislada.

Tabla de argumentos (instrumento del paso 6)

Condición del proyecto	A favor de TDD	En contra / matiz	Fuente	Peso
Requisito claro y lógica de negocio con reglas (como el descuento a clientes frecuentes, una vez aclaradas las reglas)	Los casos salen del criterio de aceptación y las pruebas guían el diseño	Si el requisito está mal entendido, las pruebas confirman el error	ISTQB (2024, p. 26); Vocke (2018)	Alto

Condición del proyecto	A favor de TDD	En contra / matiz	Fuente	Peso
Código que se mantendrá y refactorizará	Las pruebas en verde dan margen para refactorizar	Las pruebas frágiles encarecen el refactor. Fowler (2023) advierte que omitir el paso de refactor es la forma más común de hacer mal TDD	Fowler (2023); Vocke (2018)	Alto
Equipo con poca experiencia en pruebas	Obliga a pensar el diseño y la interfaz antes de codificar	Curva de aprendizaje; riesgo de pruebas mal diseñadas o aserciones débiles	Sin fuente verificada: es una inferencia del equipo (Desarrollo B, secc. 4)	Bajo
Defectos y calidad externa	Equipos reportan reducciones de defectos, a cambio de más esfuerzo inicial	Ese beneficio es un testimonio y la entrada no cita estudios. El estudio empírico no encontró efecto importante del orden test-first	Agile Alliance (s. f.); Fucci et al. (2017)	Medio
Plazos cortos o prototipo descartable	Detecta errores temprano	Cada prueba tiene costo de escritura y mantenimiento	Vocke (2018)	Medio
Interfaces, integración o rendimiento	—	La prueba unitaria no cubre esos niveles; se necesitan otras pruebas	ISTQB (2024); Vocke (2018)	Alto
Requisito con ambigüedades sin resolver (como «cliente frecuente» y «descuento correcto» en el caso de la guía)	Escribir los casos antes del código obliga a detectar las ambigüedades y preguntar temprano	Si se resuelven suponiendo, TDD produce pruebas en verde que confirman un requisito mal entendido	Desarrollo A, secc. 2 y 6; ISTQB (2024, p. 45–46)	Alto
Necesidad de trazabilidad (auditoría, cambios frecuentes del requisito)	Cada prueba nace de un criterio de aceptación, así que la trazabilidad queda incorporada (IDs CP-xx en las pruebas)	La trazabilidad también se logra con pruebas escritas después, si se documenta la matriz	Desarrollo A, secc. 5; ISTQB (2024, p. 20)	Medio

Cuándo sí

1. Hay reglas de negocio con entradas y salidas bien definidas.
2. El código va a evolucionar y se necesita refactorizar con seguridad.
3. El equipo puede trabajar en pasos pequeños. En el estudio de Fucci et al. (2017), los ciclos más cortos se asociaron con mejor calidad. Es una señal débil (un solo coeficiente significativo, p = 0,04), así que se presenta como condición favorable, no como garantía.

En todos los casos, el primer paso es aclarar el requisito y derivar casos de prueba trazables (Desarrollo A). Esa parte sí aplica a todos los proyectos.

Cuándo no (o cuándo conviene otra cosa)

1. El requisito aún se explora y cambiará por completo: conviene primero aclarar con el cliente.
2. Código desechable o prototipos de corta vida.
3. Lo que se necesita validar es la integración, la interfaz de usuario o el rendimiento, donde las pruebas unitarias aportan poco.

Contraargumento principal y respuesta

Contraargumento. «Escribir la prueba primero es lo que hace valioso a TDD, por lo tanto debería aplicarse siempre.»

Respuesta. La evidencia disponible lo matiza. En el estudio de Fucci et al. (2017), el orden en que se escriben prueba y código «had no important influence», y los resultados se asociaron con pasos pequeños y uniformes. Eso sugiere que el beneficio puede estar en el ritmo de trabajo y no en el orden. Aun así, el estudio no usa grupo de control, tiene pocas tareas y no mide mantenibilidad a largo plazo. Además, de los factores de ritmo solo la granularidad frente a la calidad es significativa al nivel de 0,05, y los modelos explican poco de la variación (R^2 ajustado de 0,10 a 0,12). No permite concluir que el test-first sea inútil. Por eso la postura es condicionada y no absoluta.

Límites de la evidencia

- Seis de las ocho fuentes son normativas o de experiencia profesional (ISTQB, ISO/IEC/IEEE 29119-1, Pressman, JUnit 5, Agile Alliance y Vocke). Fowler solo describe el ciclo. Solo una fuente es empírica (Fucci et al.).
- La demostración es pequeña e ilustrativa; no mide productividad ni calidad. El ciclo se observó con un arnés mínimo; la suite final se ejecutó con JUnit.
- El estudio empírico es observacional, con muestra por conveniencia y tareas cortas.

Qué evidencia nueva cambiaría la conclusión (pregunta final de la exposición)

- Estudios controlados con **proyectos reales y de larga duración** que midan mantenibilidad y costo, y que muestren que test-first supera a test-last con la misma granularidad. Con eso la postura se movería hacia recomendar TDD de forma más general.
- Réplicas del estudio de Fucci et al. que contradigan que el orden no importa.
- Datos del propio equipo (defectos, tiempo de corrección y costo de mantenimiento de pruebas) que mostraran que TDD no compensa en su contexto. Eso movería la postura hacia usarlo solo en las partes críticas.

Conclusiones

Qué logró la investigación

La investigación siguió un requisito desde su redacción hasta sus pruebas. El requisito «el sistema aplicará descuentos correctos a los clientes frecuentes» no se puede verificar tal como está escrito. Al identificar sus diez ambigüedades y reconstruir la cadena requisito → criterio → caso → resultado esperado, se obtuvieron doce casos trazables. La única prueba del desarrollador equivale a uno de ellos. Con ese análisis, el material de las fuentes y una demostración pequeña del ciclo rojo–verde–refactor, el equipo formuló una respuesta a la pregunta central.

Respuesta a la pregunta central

¿Es razonable defender TDD en todos los proyectos o su utilidad depende del problema, el equipo y los requisitos? Su utilidad depende de los tres. La evidencia revisada sostiene que TDD es valiosa en ciertas condiciones, pero no justifica imponerla como regla universal.

1. **Antes de discutir TDD hay que tener un requisito verificable.** En el caso de la guía, la prueba del 10 % no se puede juzgar como suficiente porque no existe un criterio de aceptación contra el cual medirla. Mientras no se defina qué es un «cliente frecuente» y un «descuento correcto», ninguna prueba, escrita antes o después del código, demuestra que el requisito se cumplió. Escribir la prueba primero tiene un efecto útil: obliga a hacer esas preguntas antes de programar.

2. **Una suite es suficiente por lo que cubre, no por cuántas pruebas tiene.** Una suite se considera suficiente cuando cubre cada criterio, cada partición y cada frontera (ISTQB, 2024). La demostración lo confirmó: al cambiar a propósito `>=` por `>`, la prueba de 99,99 siguió pasando y solo la del valor exacto 100 falló.

3. **El ciclo rojo–verde–refactor da seguridad para cambiar el código, siempre que las pruebas verifiquen comportamiento.** Si dependen de la estructura interna se vuelven frágiles y encarecen el refactor (Vocke, 2018). Si tienen aserciones débiles o les faltan casos límite, dan una falsa sensación de seguridad.

4. **El beneficio de TDD podría estar en el ritmo y no en el orden.** En el estudio de Fucci et al. (2017), el orden en que se escriben prueba y código no tuvo una influencia importante. Los ciclos más cortos se asociaron con mejor calidad, aunque con un efecto débil. Eso no prueba que los pasos pequeños sean la causa, pero sí debilita el argumento de que escribir la prueba primero es indispensable.

Postura. TDD conviene cuando:

- hay reglas de negocio con entradas y salidas definidas;
- el código se va a mantener y refactorizar;
- el equipo puede sostener la disciplina de pasos pequeños.

Conviene menos, o conviene otra práctica, cuando:

- el requisito todavía se explora y va a cambiar;
- el código es un prototipo desechable;
- lo que hay que validar es la integración, la interfaz o el rendimiento.

Qué cambia en el razonamiento del equipo

TDD cambia la pregunta con la que el equipo empieza. En lugar de «¿cómo programo esto?», la pregunta pasa a ser «¿cómo sabré que esto está bien?». En el caso de la guía, el desarrollador razonó al revés: programó, confirmó lo que ya creía y declaró terminado. La persona que propuso aclarar las reglas razonó como pide TDD: sin un resultado esperado definido no hay prueba que escribir.

Ese cambio tiene tres efectos:

- las ambigüedades aparecen antes de escribir código;
- el criterio de «terminado» deja de ser una opinión y pasa a ser un conjunto de casos que pasan;
- cada prueba queda trazada a una regla del negocio.

Ese cambio de razonamiento es lo más valioso de TDD, y también se puede aplicar sin escribir las pruebas antes del código.

Por qué más pruebas no equivale automáticamente a mejor calidad

- **Más pruebas sobre un requisito ambiguo solo multiplican la suposición inicial.** Diez pruebas del 10 % no dicen más que una si nadie confirmó que el 10 % es lo correcto.
- **Más pruebas en la misma partición casi no encuentran defectos nuevos.** Una prueba bien elegida en la frontera detecta lo que muchas en el centro dejan pasar.
- **Cada prueba tiene costo.** Vocke (2018) advierte que «every single test in your test suite is additional baggage and doesn’t come for free». Las pruebas frágiles o duplicadas consumen tiempo de mantenimiento sin proteger más.
- **Una suite en verde demuestra que pasan los casos escritos, no que no haya defectos.** La calidad la determinan la pertinencia de los casos y su trazabilidad a los criterios, no su número.

Límites de la evidencia

- **Predomina la evidencia normativa y de experiencia profesional.** Seis de las ocho fuentes (ISTQB, ISO/IEC/IEEE 29119-1, Pressman, JUnit 5, Agile Alliance y Vocke) describen o recomiendan, pero no miden resultados. Fowler aporta solo la descripción del ciclo.
- **Hay una sola fuente empírica.** Fucci et al. (2017) es un estudio observacional, sin grupo de control, con 39 profesionales de dos empresas y tres tareas cortas. Sirve para matizar la postura, no para generalizar.
- **El caso es pequeño y sus reglas son supuestas.** Las reglas de frecuencia, monto y redondeo las supuso el equipo, porque no hubo un cliente real a quien preguntar. Con otras respuestas cambian los casos, aunque no el método.

- **El ciclo no se observó con JUnit.** Las fases se observaron con un arnés mínimo en Java. La suite final sí se ejecutó con JUnit 5.9.2 (`mvn test`, 9 pruebas en verde), pero eso no repite las fases en rojo. Además, solo cubre la regla de monto, no la clasificación por frecuencia (CP-11 y CP-12).

Pendientes y qué podría mejorarse

1. Automatizar CP-11 y CP-12, la clasificación de cliente frecuente, con una segunda unidad que cuente las compras.
2. Repetir el ciclo completo con JUnit, para observar también las fases en rojo con la herramienta (la suite final ya se ejecutó con `mvn test`).
3. Verificar las citas de Fucci et al. (2017) en la versión publicada por IEEE.
4. Dejar fuera del análisis A7, A8 y A10 (promociones combinadas, exclusiones y momento de evaluación) fue una simplificación. Un análisis más completo las incluiría.
5. Para investigaciones futuras, buscar estudios controlados de larga duración que comparen test-first y test-last con la misma granularidad. Esa evidencia permitiría confirmar o cambiar esta conclusión.

Lecciones aprendidas

Lecciones de Valery Salas Vargas

Paso	Qué se aprendió	Qué dificultad apareció	Qué se mejoraría
1. Delimitación	La pregunta sobre TDD no es de sí o no: la respuesta depende del problema, del equipo y de los requisitos. Plantearlo así desde el inicio llevó a buscar evidencia a favor y en contra.	Se trabajó varios días sin tener la guía a mano, con un requisito y unas preguntas de debate supuestos. Al leer la guía hubo que rehacer el Desarrollo A, porque el caso real era otro.	Leer la ficha completa del tema antes de escribir y copiar textualmente la pregunta central, el caso y las preguntas de debate en un documento compartido.
2. Búsqueda y selección	Los datos bibliográficos se verifican contra catálogos y contra la propia fuente, no se copian del enunciado.	La guía citaba «Pressman (2012)», pero la 7.ª edición en español es de 2010. La norma ISO/IEC/IEEE 29119-1 es de pago y no se pudo leer completa.	Revisar desde el primer día si cada fuente es accesible y conseguir el ejemplar físico antes de comprometerse a citarla.
3. Lectura crítica (fuente 1)	Una fuente puede ser muy confiable para definir y poco útil para argumentar. El ISTQB define bien casi todos los conceptos, pero no dice si TDD funciona. La matriz obligó a separar lo que una fuente define de lo que demuestra.	Algunas páginas solo se pudieron tomar del índice del PDF, y hubo que corregirlas después.	Anotar cada página con el documento abierto en el momento de leer, no después.

Paso	Qué se aprendió	Qué dificultad apareció	Qué se mejoraría
4. Análisis del caso	El trabajo de prueba empieza antes de escribir casos: hay que leer el requisito buscando lo que no dice. Un requisito de una línea tuvo diez ambigüedades, y una sola prueba resultó ser uno de doce casos necesarios.	Sin un cliente real, hubo que suponer las reglas (cliente frecuente, monto mínimo y redondeo) para poder diseñar casos. También hubo que coordinar los IDs de los casos con las pruebas de Maykel.	Definir los IDs y la matriz de trazabilidad antes de repartir el trabajo, y marcar desde el inicio qué es dato del enunciado y qué es supuesto del equipo.
6. Síntesis	La postura mejora cuando junta las dos mitades: la técnica (pruebas y TDD) y la del análisis del requisito.	La primera versión de la postura era solo técnica y no incluía los argumentos del Desarrollo A. Además, el estudio empírico tenía un efecto más débil de lo que parecía al leer el resumen.	Escribir la postura entre los dos desde el inicio y leer las tablas de resultados de cada estudio, no solo su resumen.
7. Revisión cruzada	La revisión del otro encuentra lo que quien escribe ya no ve. Mi revisión encontró que las pruebas estaban en una carpeta que Maven no ejecuta y que la matriz estaba duplicada. La de Maykel encontró un error de conteo en mi resumen.	Varios *pull requests* se fusionaron antes de la aprobación del otro integrante, así que la revisión llegó tarde.	No fusionar ningún *pull request* sin la aprobación del otro integrante.
7. Uso de IA	Lo generado por IA es un borrador que hay que verificar. La responsabilidad de cada cita sigue siendo de quien la incluye.	Algunos datos, como el año y los traductores de Pressman, solo se pudieron confirmar al abrir el ejemplar, días después de citarlo.	Abrir los originales en el momento en que la IA propone una cita, y registrar en la bitácora qué se verificó y qué no.

Lecciones de Maykel Meoño González

Paso	Qué pasó	Lección
2. Búsqueda y selección	El reparto decía «Fowler, *Practical Test Pyramid*», pero el autor es Ham Vocke y el artículo solo está alojado en el sitio de Fowler. La guía de JUnit pedida ya va en la versión 6, y *TDD by Example* de Beck no se pudo abrir.	Confirmar autor, fecha y versión en la propia fuente antes de citarla, no en el reparto. Si una fuente no es accesible, buscar la alternativa que la guía permite (Agile Alliance) desde el inicio.
3. Lectura crítica (fuente 2)	Al contrastar las citas con las páginas originales, la cita de Agile Alliance no era textual y la cita 5 de Vocke estaba en otra sección. En JUnit, los números de sección cambian entre versiones (2.1 y 2.5 en la 5.9.2; 3.1 y 3.4 en la 5.3.2).	Una cita copiada de un borrador o de memoria no es una cita verificada. Hay que anotar versión y sección con la página abierta, y volver a abrir el original antes de pasar el texto al Word.

Paso	Qué pasó	Lección
3. Lectura crítica (fuente 3)	El preprint de Fucci et al. dice «We thus recommend…» y no «we recommend…». De sus modelos finales, solo el coeficiente de granularidad frente a calidad es significativo (p = 0,04).	Leer las tablas, no solo el resumen. Una recomendación puede ser razonable y estar respaldada por un efecto pequeño o no significativo; eso hay que decirlo.
5. Demostración TDD	No se pudo descargar JUnit en el entorno de preparación, así que el ciclo se observó con un arnés mínimo en Java; después `mvn test` pasó las 9 pruebas con JUnit. CP-03 pasó sin fase roja, y al cambiar `>=` por `>` solo CP-02 falló. Escribir CP-07 obligó a decidir el redondeo (ambigüedad A3).	Declarar qué se ejecutó de verdad y con qué herramienta. Una prueba que nunca estuvo en rojo no demuestra que detecte el error: hay que romper el código a propósito. Escribir la prueba primero saca a la luz las preguntas al cliente.
6. Síntesis	Una fila de la tabla de argumentos (equipos con poca experiencia) citaba una fuente que no la respaldaba. Se cambió a «sin fuente verificada» y peso bajo.	Cada fila de la postura debe poder rastrearse a una cita verificada. Lo que es inferencia del equipo se dice como inferencia.
7. Revisión cruzada	Los errores que más pesaron (autoría de Vocke, carpeta de pruebas que Maven no ejecuta, matriz duplicada) los encontró la revisión del otro integrante.	La revisión cruzada encuentra lo que quien escribe ya no ve. Hay que pedirla antes de fusionar, no después.
7. Uso de IA	La IA contrastó citas y generó borradores rápido, pero leyó algunas páginas resumidas, no pudo abrir IEEE ni el libro de Beck y no pudo ejecutar JUnit.	La IA acelera, pero cada cita sigue siendo responsabilidad de quien la incluye: hay que abrir los originales y correr las pruebas uno mismo.

Referencias

Agile Alliance. (s. f.). *TDD*. En *Agile glossary*. Recuperado el 11 de noviembre de 2026, de https://agilealliance.org/glossary/tdd/

Bechtold, S., Brannen, S., Link, J., Merdes, M., Philipp, M., de Rancourt, J., & Stein, C. (s. f.). *JUnit 5 user guide* (Versión 5.9.2) [Documentación técnica]. JUnit. https://docs.junit.org/5.9.2/user-guide/

Fowler, M. (2023, 11 de diciembre). *Test driven development*. martinfowler.com. https://martinfowler.com/bliki/TestDrivenDevelopment.html

Fucci, D., Erdogmus, H., Turhan, B., Oivo, M., & Juristo, N. (2017). A dissection of the test-driven development process: Does it really matter to test-first or to test-last? *IEEE Transactions on Software Engineering, 43*(7), 597–614. https://doi.org/10.1109/TSE.2016.2616877

International Organization for Standardization, International Electrotechnical Commission, & Institute of Electrical and Electronics Engineers. (2022). *Software and systems engineering — Software testing — Part 1: General concepts* (ISO/IEC/IEEE 29119-1:2022). https://webstore.iec.ch/publication/74072

International Software Testing Qualifications Board. (2024). *Certified Tester Foundation Level syllabus* (Versión 4.0.1). https://www.istqb.org/wp-content/uploads/2024/11/ISTQB_CTFL_Syllabus_v4.0.1.pdf

Pressman, R. S. (2010). *Ingeniería del software: Un enfoque práctico* (V. Campos Olguín & J. Enríquez Brito, Trads.; 7.ª ed.). McGraw-Hill.

Vocke, H. (2018, 26 de febrero). *The practical test pyramid*. martinfowler.com. https://martinfowler.com/articles/practical-test-pyramid.html

Declaración de uso de inteligencia artificial

En la elaboración de este informe ambos integrantes usaron un asistente de inteligencia artificial generativa, **Claude (Anthropic)**. Lo usaron para:

- localizar y verificar datos bibliográficos;
- organizar el reparto de tareas;
- redactar borradores de secciones;
- revisar la coherencia entre las partes del informe.

El detalle de cada uso (herramienta, modelo, fecha, instrucción y qué se aprovechó) está en el **Anexo C, Bitácora de uso de IA**.

La herramienta no sustituyó la verificación de las fuentes. Las citas se contrastaron con las versiones disponibles en línea de cada documento:

- el programa del ISTQB (PDF oficial v4.0.1);
- la guía de usuario de JUnit 5.9.2;
- las páginas de Agile Alliance, Vocke y Fowler;
- el preprint de Fucci et al. en arXiv, porque no hubo acceso a la versión publicada por IEEE.

La norma ISO/IEC/IEEE 29119-1 es de pago: solo se citó su alcance, tomado de la ficha oficial de la norma. El libro de Pressman se verificó con un ejemplar digital de la 7.ª edición: la página legal (año © 2010, edición y traductores) y las citas de las páginas 385 y 425. Lo que no se pudo verificar quedó señalado en el texto.

Los borradores generados se revisaron, se corrigieron y se adaptaron. La revisión cruzada se hizo por *pull requests* en un repositorio compartido. Varios se fusionaron antes de la aprobación formal del otro integrante, y la revisión se completó después, con una ronda de correcciones.

Los autores asumen la responsabilidad total por el contenido, las interpretaciones y las conclusiones del informe.

Anexos

Actualizado el 13-nov-2026. Los anexos A y B son copias de `fuentes/matriz-lectura-critica.md` y de la tabla del apartado «Proceso de planificación»: si esos archivos cambian, hay que volver a copiarlos aquí.

Anexo A. Matriz de lectura crítica

Tipo de evidencia:

- **N**: normativa o consenso de un organismo
- **P**: experiencia profesional u opinión experta
- **E**: empírica (estudio con datos)

Matriz en el formato de la guía

Esta tabla es la que exige la guía (sección 4, «Anexo obligatorio»). Las secciones que siguen la detallan fuente por fuente, con las citas textuales y sus páginas.

Fuente y tesis	Evidencia consultada	¿Qué no demuestra / límite?	Interpretación de la pareja
Fuente 1 · ISTQB (2024). Las pruebas se diseñan a partir de una base de prueba trazable (requisitos y criterios de aceptación), con técnicas como la partición de equivalencia y los valores límite. TDD es un enfoque *test-first* que aplica el principio de probar temprano.	Programa oficial v4.0.1, consenso de expertos de los comités nacionales del ISTQB. Secciones 1.4.4 (trazabilidad), 2.1.3 (TDD, ATDD y BDD), 2.2.1 (prueba de componente), 4.2 (técnicas de caja negra) y 4.5 (criterios de aceptación).	No aporta datos sobre si TDD mejora la calidad o la productividad. Presenta TDD de forma neutral, sin decir cuándo no conviene. Es un programa de certificación, por eso simplifica.	Es la fuente más confiable para definir y diseñar casos: sostiene todo el Desarrollo A. No sirve para decidir si TDD conviene. Por eso hizo falta una fuente empírica.
Fuente 2 · Vocke (2018). Las pruebas unitarias deben ser la base de la estrategia de pruebas. Hay que probar el comportamiento observable y no la estructura interna, y cada prueba tiene un costo de mantenimiento. Recomienda TDD.	Artículo técnico basado en la experiencia profesional del autor, con ejemplos en Java (microservicios), publicado en martinfowler.com. Secciones *Unit tests*, *Test Structure*, *What to Test?* y *Avoid Test Duplication*.	Es opinión experta de un solo autor, sin datos ni método de investigación. Recomienda TDD sin discutir cuándo no conviene.	Respalda la estructura AAA y la explicación de las pruebas frágiles (Desarrollo B). Su advertencia de que cada prueba «doesn’t come for free» sostiene que más pruebas no equivalen a más calidad.

Fuente y tesis	Evidencia consultada	¿Qué no demuestra / límite?	Interpretación de la pareja
Fuente 3 · Fucci et al. (2017). Escribir la prueba antes o después del código no explica, de forma importante, la calidad ni la productividad. Los beneficios atribuidos a TDD vendrían de trabajar en pasos pequeños y uniformes.	Estudio observacional con 39 profesionales de dos empresas y 82 datos, analizados con regresión múltiple. Mide granularidad, uniformidad, orden y esfuerzo de refactorización. Contrastado con el preprint de arXiv.	No prueba causalidad: no hay aleatorización ni grupo de control. Usa tareas cortas y requisitos fijos. Los modelos explican poco (R^2 ajustado de 0,10 a 0,12) y solo la granularidad frente a la calidad es significativa entre los factores de ritmo.	Matiza la postura: el valor de TDD podría estar en el ritmo de trabajo y en obligar a aclarar el requisito, no en el orden. No alcanza para afirmar que *test-first* sea inútil ni para generalizar.

Fuente 1 · ISTQB (2024), *Certified Tester Foundation Level syllabus* v4.0.1 · Responsable: Valery

Ficha general

Campo	Contenido
Autor / organismo	International Software Testing Qualifications Board, organismo internacional sin fines de lucro que define la certificación de testers
Tipo de documento	Programa (*syllabus*) de certificación de nivel básico. El documento se describe así: «This syllabus forms the basis for the International Software Testing Qualification at the Foundation Level» (p. 10)
Tipo de evidencia	**N**: consenso de expertos del sector, revisado por los comités nacionales del ISTQB
Vigencia	v4.0 del 21-04-2023; v4.0.1 de mantenimiento del 15-09-2024
Acceso	PDF gratuito y oficial en istqb.org

Afirmaciones que usa el informe

#	Sección (pág.)	Cita textual	Idea en nuestras palabras	Uso en el informe
1	1.4.4 (p. 20)	«To implement effective test monitoring and test control, it is important to establish and maintain traceability throughout the test process between the test basis elements…»	La trazabilidad entre la base de prueba (requisitos, criterios) y los casos hace posible controlar las pruebas y saber qué está cubierto.	Desarrollo A: matriz requisito → criterio → caso → resultado
2	2.1.3 (p. 26)	TDD: «Directs the coding through test cases (instead of extensive software design)»	En TDD, las pruebas guían la escritura del código, en lugar de un diseño previo extenso.	Introducción; puente con el Desarrollo B

#	Sección (pág.)	Cita textual	Idea en nuestras palabras	Uso en el informe
3	2.1.3 (p. 26)	«Each of these approaches implements the principle of early testing (see section 1.3) and follows shift left…»	TDD, ATDD y BDD llevan las pruebas a las primeras etapas del desarrollo.	Introducción (contexto); postura sobre TDD
4	2.1.3 (p. 26)	ATDD: «Derives tests from acceptance criteria as part of the system design process»	Las pruebas de aceptación se derivan de los criterios de aceptación.	Desarrollo A: del criterio de aceptación al caso
5	2.2.1 (p. 29)	«Component testing (also known as unit testing) focuses on testing components in isolation.»	La prueba unitaria es la prueba de un componente aislado.	Definición de prueba unitaria; Desarrollo B
6	4.2.1 (p. 39)	«Therefore, one test for each partition is sufficient.»	Si los valores de una partición se comportan igual, basta con un caso por partición, válida o inválida.	Desarrollo A: casos normales y de error
7	4.2.2 (p. 40–41)	2 valores: «for each boundary value there are two coverage items: this boundary value and its closest neighbor»; 3 valores: «…three coverage items: this boundary value and both its neighbors.»	Se prueban los valores en cada frontera de las particiones y sus vecinos. El análisis de 3 valores es más riguroso que el de 2.	Desarrollo A: casos límite del descuento del 10 %
8	4.4.1 (p. 43)	«Error guessing is a test technique used to anticipate the occurrence of errors, defects, and failures…»	La experiencia del tester sirve para anticipar errores típicos, como entradas vacías, negativas o con un formato incorrecto.	Desarrollo A: casos de error
9	4.5.2 (p. 45–46)	Los criterios de aceptación sirven para «Define the scope of the user story» y «Serve as a basis for the user story acceptance testing»	El criterio de aceptación delimita el requisito y es la base de sus pruebas.	Desarrollo A; preguntas de debate 1 y 2 (ambigüedades)

#	Sección (pág.)	Cita textual	Idea en nuestras palabras	Uso en el informe
10	4.5.3 (p. 46)	«ATDD is a test-first approach…»	ATDD también escribe las pruebas antes del código, pero en el nivel de aceptación.	Desarrollo A y B (relación entre ATDD y TDD)

Valoración crítica

Fortalezas	**Lo produce un organismo internacional y lo revisan expertos de muchos países. Es el vocabulario estándar del sector, es gratuito y es reciente (2023–2024). Las definiciones son precisas y numeradas, así que se pueden citar con exactitud.**
Limitaciones	Es un programa de certificación: describe y define, pero **no aporta evidencia empírica** sobre si TDD mejora la calidad o la productividad. Presenta TDD de forma neutral y no dice cuándo no conviene. Su objetivo es formar para un examen, por eso simplifica.
Posibles sesgos	Su interés es promover la certificación y la profesión del tester. Las técnicas que incluye se presentan como buenas prácticas generales, sin considerar el contexto del proyecto.
Qué no puede responder	Si TDD conviene en *todos* los proyectos. Para eso hacen falta las fuentes de Maykel (Beck o Agile Alliance, Fowler) y, de ser posible, un estudio empírico.
Confiabilidad para el informe	Alta para definiciones y técnicas de diseño de casos. Baja como argumento a favor o en contra de TDD.

Páginas tomadas del índice y de las marcas de página del PDF v4.0.1 (78 p.).

Fuente 2 · Vocke (2018), *The practical test pyramid* · Responsable: Maykel

Ficha general

Campo	Contenido
Autor / organismo	Ham Vocke, ingeniero de software. Publicado en el sitio de Martin Fowler (ThoughtWorks)
Tipo de documento	Artículo técnico en línea, 26-feb-2018
Tipo de evidencia	**P**: experiencia profesional, sin datos ni método de investigación
Vigencia	2018. Los principios de las pruebas unitarias no han cambiado, pero los ejemplos de herramientas sí
Acceso	Gratuito, https://martinfowler.com/articles/practical-test-pyramid.html

Afirmaciones que usa el informe

#	Sección	Cita textual	Idea en nuestras palabras	Uso en el informe
1	Unit tests	«The foundation of your test suite will be made up of unit tests.»	Las pruebas unitarias son la base de la suite.	Desarrollo B: lugar de la prueba unitaria
2	Unit tests › Test Structure	«A good structure for all your tests (this is not limited to unit tests) is this one:» con tres pasos: «Set up the test data», «Call your method under test» y «Assert that the expected results are returned». El artículo agrega: «There's a nice mnemonic to remember this structure: "Arrange, Act, Assert".»	Una estructura fija de tres partes para toda prueba, que el propio artículo llama Arrange–Act–Assert.	Desarrollo B: AAA
3	Unit tests › Mocking and Stubbing	«A lot of people use the terms Mock and Stub interchangeably.»	Los dobles de prueba se confunden; hay que definirlos.	Desarrollo B: aislamiento
4	Unit tests › What to Test?	«Don't reflect your internal code structure within your unit tests.» · «Test for observable behaviour instead.»	Probar comportamiento observable evita pruebas frágiles.	Desarrollo B: pruebas frágiles
5	Unit tests › Mocking and Stubbing	«Look into Test-Driven Development and let your unit tests guide your development»	Recomienda TDD como guía del desarrollo.	Postura sobre TDD
6	Avoid Test Duplication	«Every single test in your test suite is additional baggage and doesn't come for free.»	Cada prueba tiene costo de mantenimiento.	Postura sobre TDD (contraargumento)

Valoración crítica

Fortalezas	**Claro, práctico y bien estructurado. Está en un sitio de referencia del sector y es de acceso libre. Cubre justo los temas del Desarrollo B (aislamiento, estructura, fragilidad).**
Limitaciones	Es opinión experta con un solo autor, sin estudios que respalden las recomendaciones. No define el ciclo rojo–verde–refactor ni trata la «falsa sensación de seguridad» con esos términos. Sus ejemplos están en un contexto concreto (microservicios en Java).
Posibles sesgos	Recomienda TDD sin discutir cuándo no conviene. Parte de la experiencia de un equipo de producto con prácticas ágiles maduras.
Qué no puede responder	Si TDD mejora la calidad o la productividad con datos. Tampoco dice en qué proyectos no conviene.
Confiabilidad para el informe	Media-alta para buenas prácticas de pruebas unitarias. Baja como prueba de que TDD «funciona».

Fuente 3 · Fucci et al. (2017) · Responsables: ambos

Ficha general

Campo	Contenido
Autor / organismo	Davide Fucci, Hakan Erdogmus, Burak Turhan, Markku Oivo y Natalia Juristo (investigación académica)
Tipo de documento	Artículo científico, *IEEE Transactions on Software Engineering* (2017)
Tipo de evidencia	**E**: estudio observacional con datos y regresión múltiple
Diseño	39 profesionales de dos empresas, 82 datos completos, tres tareas (Mars Rover, Bowling Scorekeeper y MusicPhone)
Acceso	Preprint gratuito en arXiv (1611.05994); versión publicada por IEEE

Afirmaciones que usa el informe (citas contrastadas el 11-nov-2026 con el preprint de arXiv 1611.05994; no se contrastaron con la versión publicada por IEEE. Volumen 43(7) y páginas 597–614 según un índice bibliográfico) y páginas 597–614 confirmados en un índice bibliográfico)

#	Cita textual	Idea en nuestras palabras	Uso en el informe
1	(Resumen, p. 597 en IEEE) «Sequencing, the order in which test and production code are written, had no important influence.»	En el estudio, escribir la prueba antes o después no explicó mejor la calidad ni la productividad.	Postura sobre TDD; contraargumento

#	Cita textual	Idea en nuestras palabras	Uso en el informe
2	(Resumen, p. 597 en IEEE) «The claimed benefits of TDD may not be due to its distinctive test-first dynamic»	Los beneficios atribuidos a TDD podrían venir de otra parte.	Postura sobre TDD
3	(Resumen, p. 597 en IEEE) «TDD-like processes encourage fine-grained, steady steps that improve focus and flow.»	TDD ayuda porque obliga a dar pasos pequeños y parejos.	Postura sobre TDD (cuándo sí)
4	(Secc. VIII, *Discussion*) «We thus recommend focusing on breaking down development tasks into as small and as uniform steps as possible.»	La recomendación práctica es trabajar en pasos pequeños y uniformes.	Conclusión

Valoración crítica

Fortalezas	**Es la única fuente con datos. Usa profesionales y no solo estudiantes. Mide por separado cuatro factores del proceso (granularidad, uniformidad, orden y esfuerzo de refactor).**
Limitaciones	Sin aleatorización ni grupo de control, así que no prueba causalidad. Muestra por conveniencia, solo dos empresas y tres tareas (dos artificiales) con requisitos fijos. Tareas cortas. Los modelos finales explican una parte pequeña de la variación (R^2 ajustado de 0,12 para calidad y 0,10 para productividad). De los factores de ritmo, solo la granularidad frente a la calidad es significativa al nivel de 0,05 ($p = 0,04$). La uniformidad frente a la calidad ($p = 0,09$) y la granularidad y la uniformidad frente a la productividad ($p =$ 0,11 y 0,08) no lo son. El esfuerzo de refactorización sí es significativo en los dos modelos (calidad $p = 0,03$; productividad $p = 0,02$). El orden test-first no entra en ningún modelo final. Por eso la recomendación de pasos pequeños y uniformes es una señal y no una prueba.
Posibles sesgos	Participantes voluntarios o elegidos por sus empresas. Experiencia sesgada hacia perfiles medios y senior.
Qué no puede responder	El efecto de largo plazo en mantenibilidad, ni el efecto en proyectos grandes con requisitos cambiantes.
Confiabilidad para el informe	Media: sirve para matizar la postura (el orden test-first no sería lo decisivo), no para generalizar.

Nota técnica del propio preprint: el texto llama «positivas» a las asociaciones de granularidad y uniformidad, pero los coeficientes reportados son negativos (ciclos más cortos y parejos se asocian con mejores resultados). Conviene leerlo con cuidado al citar.

Anexo B. Registro de responsabilidades

Estado final de cada tarea al 13 de noviembre de 2026.

#	Tarea	Responsable	Revisa	Fecha límite	Estado
1	Buscar y verificar 3 fuentes: ISTQB CTFL v4.0, ISO/IEC/IEEE 29119, Pressman	Valery	Maykel	mar 3 nov	Hecha
2	Buscar y verificar 3 fuentes: JUnit 5 User Guide, TDD (Agile Alliance), Vocke *Practical Test Pyramid*	Maykel	Valery	mar 3 nov	Hecha. Citas contrastadas con los originales en línea el 11-nov. Fucci et al. se contrastó con el preprint de arXiv, no con IEEE Xplore
3	Metodología (7 pasos, con instrumentos y criterios)	Valery	Maykel	mié 4 nov	Hecha
4	Proceso de planificación	Maykel	Valery	mié 4 nov	Hecha (estados actualizados el 13-nov)
5	Matriz de lectura crítica	Ambos	Ambos	sáb 7 nov	Hecha. Incluye la tabla en el formato de la guía
6	Introducción	Valery	Maykel	sáb 7 nov	Hecha
7	Desarrollo A	Valery	Maykel	lun 9 nov	Hecha
8	Desarrollo B	Maykel	Valery	lun 9 nov	Hecha (ajustada al caso de clientes frecuentes el 13-nov). El ciclo se observó con un arnés mínimo en Java; el 13-nov `mvn test` pasó las 9 pruebas con JUnit 5.9.2
9	Postura condicionada sobre TDD	Ambos	Ambos	mar 10 nov	Hecha. Está en el informe con los argumentos de ambos
10	Resumen ejecutivo	Valery	Maykel	mar 10 nov	Hecha

#	Tarea	Responsable	Revisa	Fecha límite	Estado
11	Abstract	Maykel	Valery	mar 10 nov	Hecha (cifras alineadas con el resumen el 13-nov)
12	Conclusiones	Valery	Maykel	mar 10 nov	Hecha
13	Lecciones aprendidas	Ambos	Ambos	mié 11 nov	Hecha. Cada integrante escribió las de los pasos que ejecutó
14	Referencias APA y declaración de uso de IA	Valery	Maykel	mié 11 nov	Hecha
15	Anexos	Maykel	Valery	mié 11 nov	Hecha. Anexos A a E
16	Formato Word	Maykel	Valery	jue 12 nov	Hecha el 13-nov: portada, índice automático, encabezado y número de página sin numerar la portada
17	Tabla de correspondencia y PDF	Ambos	Ambos	jue 12 nov	En curso: falta la revisión conjunta del PDF final

Anexo C. Bitácora de uso de IA

Integrante	Herramienta y modelo	Fecha	Prompt (resumen)	Qué se aprovechó
Maykel Meoño González	Claude (Anthropic), modelo configurado `claude-sonnet-5-5`	6-nov-2026	«Analiza el zip y el docx y dime qué labores pendientes tiene Maykel»	Lista de tareas pendientes de Maykel, según el reparto y el estado del repositorio
Maykel Meoño González	Claude (Anthropic), modelo configurado `claude-sonnet-5-5`	6-nov-2026	«Haz las tareas correspondientes al informe»	Borradores de: fichas de las fuentes 2 y 3, planificación, Desarrollo B, postura sobre TDD, abstract, lecciones aprendidas, anexos, proyecto de demostración JUnit y plantilla Word.

Integrante	Herramienta y modelo	Fecha	Prompt (resumen)	Qué se aprovechó
Maykel Meoño González	Claude (Anthropic), modelo configurado `claude-sonnet-5-5`	11-nov-2026	«Realiza las actividades únicamente asignadas a Maykel hasta el miércoles 11 de noviembre; todavía no le hagas nada a la presentación; ignora lo marcado para ambos o para Valery»	Contraste de las citas con las páginas originales en línea y corrección de las fichas de fuentes (tarea 2); actualización del proceso de planificación (tarea 4); ajuste del Desarrollo B a los casos CP-01 a CP-09, con la recreación del proyecto `demo-tdd/` y los resultados del ciclo observados con un arnés sin JUnit (tarea 8); abstract alineado con el resumen y las conclusiones (tarea 11); anexos A a E y la figura 1 (tarea 15).
Maykel Meoño González	Claude (Anthropic), modelo configurado `claude-sonnet-5-5`	13-nov-2026	«Verifica si todas las tareas asignadas únicamente a Maykel están completas» y «Haz las correcciones correspondientes»	Revisión del estado de las tareas de Maykel; ajuste del Desarrollo B, el abstract, los anexos y la planificación al caso de clientes frecuentes; armado del documento Word final.
Valery Salas Vargas	Claude (Anthropic), modelo configurado `claude-opus-5-5`	1-nov-2026	«Buscar y verificar ISTQB CTFL v4.0, ISO/IEC/IEEE 29119-1 y Pressman (2012)»	Datos bibliográficos, referencias APA y mapeo de secciones de las tres fuentes (tarea 1)
Valery Salas Vargas	Claude (Anthropic), modelo configurado `claude-opus-5-5`	6-nov-2026	«Continúa con mi siguiente tarea» (metodología, matriz de la fuente 1, introducción, Desarrollo A)	Borradores de la metodología, la matriz de la fuente 1, la introducción, el Desarrollo A y las filas del Desarrollo A en la postura; revisión del aporte de Maykel. Valery revisó los borradores y los aprobó.

Integrante	Herramienta y modelo	Fecha	Prompt (resumen)	Qué se aprovechó
Valery Salas Vargas	Claude (Anthropic), modelo configurado `claude-opus-5-5`	11-nov-2026	«Empieza a trabajar en lo mío»	Borradores del resumen ejecutivo, las conclusiones, las referencias APA, la declaración de uso de IA, las lecciones aprendidas (parte de Valery) y el contenido de las diapositivas 1–7 y 12–13. Valery revisó los borradores y los aprobó.

Verificación de lo generado por IA.

- Las citas de las fuentes de Maykel (Vocke, Agile Alliance, Fowler, JUnit 5.9.2 y el preprint de Fucci et al.) se contrastaron el 11-nov-2026 con las páginas originales en línea. La lectura de las páginas la hizo la herramienta, que a veces recibe una versión resumida del texto. Cada integrante debe abrir los originales de las fuentes que cita (regla 2).
- No se pudo abrir *TDD by Example* de Beck ni IEEE Xplore. El volumen y las páginas de Fucci et al. (2017) se confirmaron en un índice bibliográfico y falta contrastarlos con IEEE.
- JUnit no se pudo descargar en el entorno de preparación de la herramienta de IA. El ciclo del Anexo E se observó con un arnés mínimo en Java. Maykel corrió después `mvn test` en su computadora (13-nov-2026, 03:21): 9 pruebas ejecutadas, 0 fallas, BUILD SUCCESS.
- Las páginas del ISTQB: p. 26 (TDD) y p. 29 (componente) se confirmaron; la sección 4.2.2 empieza en la p. 40 y la frase sobre los errores en las fronteras está en la p. 41, por lo que la matriz anota p. 40–41.

Anexo D. Figuras y su fuente

Figura	Descripción	Fuente
Figura 1	Ciclo rojo–verde–refactor	Elaboración propia, con base en Fowler (2023)

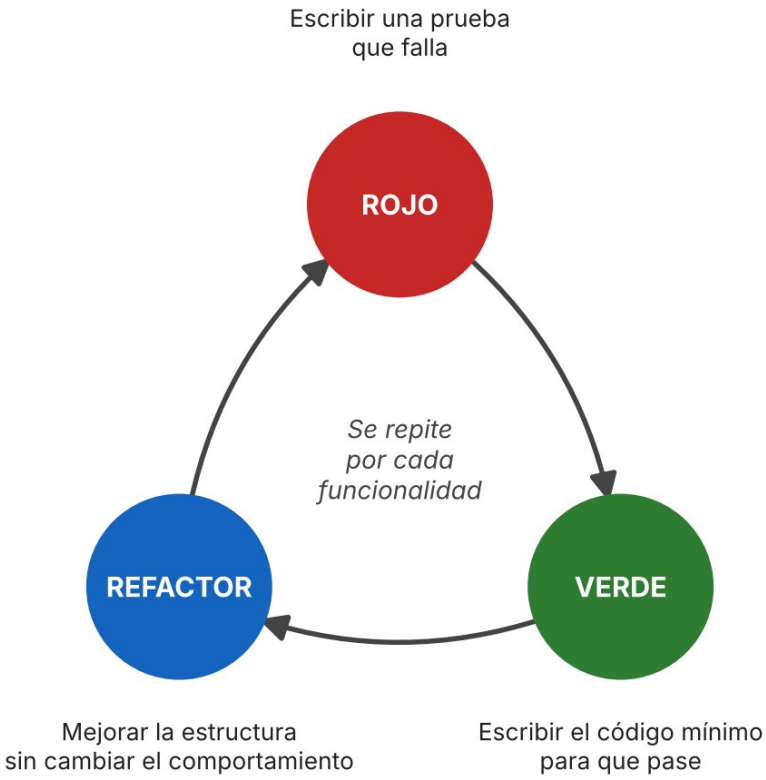


Figura 1. Ciclo rojo–verde–refactor. Elaboración propia, con base en Fowler (2023).

Anexo E. Código y bitácora de la demostración

Proyecto Maven `demo-tdd/` (JUnit 5.9.2, Java 17). Para ejecutarlo: `cd demo-tdd` y `mvn test`.

Clase probada (`CalculadoraDescuento.java`):

```
package cr.utn.descuento;

/**
 * R-01: El sistema aplica un descuento del 10 % a las compras de clientes
frecuentes
 * cuando el monto es de 100 o más.
 * Supuestos (ver Desarrollo A): 100 sí recibe descuento (A1), los negativos
 * son inválidos (A7) y el total se redondea a dos decimales (A3, CA-04).
 */
public class CalculadoraDescuento {

    static final double UMBRAL = 100.0;
    static final double TASA_DESCUENTO = 0.10;

    public double calcularTotal(double monto) {
        if (monto < 0) {
```

```
            throw new IllegalArgumentException("El monto no puede ser
negativo");
        }
        double total = monto >= UMBRAL ? monto * (1 - TASA_DESCUENTO) :
monto;
        return redondearADosDecimales(total);
    }

    private double redondearADosDecimales(double valor) {
        return Math.round(valor * 100.0) / 100.0;
    }
}
```

Pruebas (`CalculadoraDescuentoTest.java`): una por cada caso CP-01 a CP-09 de la matriz de trazabilidad del Desarrollo A.

```
package cr.utn.descuento;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertThrows;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;

/** Casos CP-01 a CP-09 de la matriz de trazabilidad (Desarrollo A, secc. 5).
*/
class CalculadoraDescuentoTest {

    private static final double TOLERANCIA = 0.0001;

    private CalculadoraDescuento calculadora;

    @BeforeEach
    void preparar() {                       // Arrange compartido: objeto
nuevo en cada prueba
        calculadora = new CalculadoraDescuento();
    }

    @Test
    @DisplayName("CP-01 normal: monto por debajo del umbral no recibe
descuento")
    void cp01MontoNormalSinDescuento() {
        double monto = 50.0;                                        //
Arrange
        double total = calculadora.calcularTotal(monto);            // Act
        assertEquals(50.0, total, TOLERANCIA);                      //
Assert
    }

    @Test
    @DisplayName("CP-02 límite: monto exacto en el umbral recibe 10 %")
    void cp02MontoEnElUmbralRecibeDescuento() {
```

```
        double total = calculadora.calcularTotal(100.0);
        assertEquals(90.0, total, TOLERANCIA);
    }

    @Test
    @DisplayName("CP-03 límite: justo debajo del umbral no recibe descuento")
    void cp03MontoJustoDebajoDelUmbral() {
        double total = calculadora.calcularTotal(99.99);
        assertEquals(99.99, total, TOLERANCIA);
    }

    @Test
    @DisplayName("CP-04 error: monto negativo lanza excepción")
    void cp04MontoNegativoLanzaExcepcion() {
        assertThrows(IllegalArgumentException.class, () ->
calculadora.calcularTotal(-1.0));
    }

    @Test
    @DisplayName("CP-08 límite/error: -0,01 lanza excepción")
    void cp08JustoDebajoDeCeroLanzaExcepcion() {
        assertThrows(IllegalArgumentException.class, () ->
calculadora.calcularTotal(-0.01));
    }

    @ParameterizedTest(name = "{0}: monto {1} -> total {2}")
    @CsvSource({
        "CP-05, 200.00, 180.00",
        "CP-06, 0.00,   0.00",
        "CP-07, 100.01, 90.01",
        "CP-09, 0.01,   0.01"
    })
    @DisplayName("CP-05, CP-06, CP-07 y CP-09: varios valores con el mismo
molde AAA")
    void variosCasos(String id, double monto, double esperado) {
        assertEquals(esperado, calculadora.calcularTotal(monto), TOLERANCIA);
    }
}
```

Tabla del ciclo (resultados observados el 11-nov-2026 con un arnés mínimo sin JUnit; la suite final pasó con JUnit el 13-nov-2026; ver el archivo `anexos/bitacora-ciclo-tdd.md` para la nota completa).

#	Prueba	Fase	Cambio hecho	Resultado esperado	Resultado real (arnés)	Observación
1	CP-01 (50 → 50)	Rojo	Escribir la prueba sin la clase	No compila	`error: cannot find symbol … class CalculadoraDescuento`	En un lenguaje compilado, que no compile cuenta como rojo

#	Prueba	Fase	Cambio hecho	Resultado esperado	Resultado real (arnés)	Observación
2	CP-01	Verde	`return monto;`	Pasa	CP-01 pasa	Implementación mínima
3	CP-02 (100 → 90)	Rojo	Escribir la prueba	Falla: se obtiene 100, se esperaba 90	CP-02 falla: «obtuvo 100.0, esperaba 90.0»	CP-01 sigue pasando
4	CP-02	Verde	`return monto >= 100 ? monto * 0.9 : monto;`	Pasa; CP-01 sigue pasando	CP-01 y CP-02 pasan	
5	CP-03 (99,99 → 99,99)	Verde sin rojo	Solo escribir la prueba	Pasa de inmediato	CP-03 pasa sin cambiar el código	No hubo fase roja
5b	CP-02 (mutante)	Comprobación	Cambiar `>=` por `>`	CP-02 falla	CP-02 falla: «obtuvo 100.0, esperaba 90.0»	Muestra que CP-02 detecta el error de frontera. CP-03 no lo detecta
6	CP-04 (−1 → excepción)	Rojo	Escribir la prueba	Falla: no lanza excepción	CP-04 falla: «no lanzó excepción; devolvió -1.0»	
7	CP-04	Verde	Agregar `if (monto < 0) throw …`	Pasa	CP-01 a CP-04 pasan	
8	CP-07 (100,01 → 90,01)	Rojo	Escribir la prueba (criterio CA-04: redondeo a 2 decimales)	Falla: se obtiene 90,009	CP-07 falla: «obtuvo 90.009, esperaba 90.01»	Supuesto A3 por confirmar con el cliente
9	CP-07	Verde	`Math.round (total * 100.0) / 100.0`	Pasa	CP-01 a CP-04 y CP-07 pasan	
10	CP-01 a CP-09	Refactor	`100` y `0.10` pasan a constantes `UMBRAL` y `TASA_DESCUENTO`; el redondeo pasa a un método privado	Las 9 pruebas siguen en verde	CP-01 a CP-09 pasan (CP-05, CP-06, CP-08 y CP-09 se agregan ya con la clase final y pasan de inmediato)	CP-05, CP-06, CP-08 y CP-09 tampoco pasaron por rojo

#	Prueba	Fase	Cambio hecho	Resultado esperado	Resultado real (arnés)	Observación
11	Prueba frágil (ejemplo)	—	Con Mockito, verificar que se llama `aplicarTasa()`; luego refactorizar	Se rompe sin cambiar el comportamiento	**No se ejecutó**	Opcional; requiere agregar Mockito. Se explica en el Desarrollo B, sección 3, sin resultado observado

Lo que la bitácora muestra y lo que no

- Se observó un ciclo completo de rojo, verde y refactor (pasos 3 a 4, 6 a 7 y 8 a 10), con dos pruebas que no pasaron por rojo (CP-03 y, al final, CP-05, CP-06, CP-08 y CP-09).
- El mutante del paso 5b muestra que la frontera de 100 solo la protege CP-02. CP-03 pasa tanto con `>=` como con `>`.
- No se midió tiempo ni productividad. Es una demostración ilustrativa, no evidencia sobre TDD.